

Technical Research Report on NBAccess, ClaudeRuntime, ClaudeOrchestrator, and SourceVault

Research reference date: July 16, 2026 (Final Edition)

Executive Summary

The four public GitHub repositories examined in this report - NBAccess, ClaudeRuntime, ClaudeOrchestrator, and SourceVault - together form a layered system built on Wolfram Language/Mathematica. NBAccess provides the final boundary for Notebook access, confidentiality, and safe execution; ClaudeRuntime manages single-LLM turns and the Expression-Proposal loop; ClaudeOrchestrator coordinates multiple workers, workflows, and a single committer; and SourceVault manages source lifecycles, evidence, search, and long-term knowledge.

Layer	Primary responsibility	Representative boundary
NBAccess	Notebook semantic API, privacy, held-expression validation, cryptographic keys	Final boundary for execution and secrets
ClaudeRuntime	Single-agent turns, provider adapters, external runners, task placement	Proposals are not directly evaluated as commands
ClaudeOrchestrator	Planning, workers, Petri-net workflows, reduction, commit	Workers produce artifacts; the Notebook has a single committer
SourceVault	Source/snapshot/claim management, URIs, search, MCP, Knowledge Home	Knowledge sharing while preserving release gates and provenance

In practical use, however, these four layers are not the whole story. Although it is less visible at the architectural level, `transreal/claudecode` still contains many foundational functions, including ClaudeEval, multi-provider integration, Claude Directives, the shared polling tick, palette integration, and MCP server registration. It therefore remains important as a compatibility and glue layer supporting the current integrated environment. Even as responsibilities migrate toward ClaudeRuntime and ClaudeOrchestrator, understanding present-day operation still requires treating `claudecode` as background infrastructure.

The system is also not limited to the simple pattern of “use ClaudeEval to propose a Wolfram expression from a prompt, validate it with NBAccess, and execute it.”

SourceVault PromptRouter can resolve saved or deterministic PromptRoutes before any LLM call and can use ClaudeEval to propose unevaluated expressions. Complex prompts can be handled by ClaudeOrchestrator PromptWorkflow as WorkflowNet candidates; even after safe parsing and validation, newly generated workflows stop in NeedsApproval. SourceVault also implements spec-review, which builds consensus on a design specification from Notebook requirements, and spec-impl, which generates and verifies coded workflows from an approved specification. In other words, the development process itself - requirements -> specification -> implementation -> verification -> catalog registration - has been turned into a workflow.

SourceVault is moving toward representing ingested sources not merely as file paths but through canonical `sv://...` URIs in external APIs, search results, and relationship descriptions. Media such as mail, web, and PDF are represented as sidecar metadata (`Class`, `MediaType`, `Kind`) rather than as URI namespaces, while identity namespaces such as `object`, `snapshot`, `artifact`, `chunk`, `file`, `packageapi`, and `directive` are used for stable object identity. Legacy references such as `sourceId`, `snapshot:...`, and `blob:...` remain for compatibility, but across MCP and Orchestrator boundaries, `sv://` URIs serve as stable opaque references.

SourceVault MCP safely exposes this URI layer to external LLMs. The current MCP surface provides 26 tools covering catalog/search/get, access requests, runtime capabilities, web search, mail and OOPS search, GitHub commit logs, Claude Directives rules/skills, and append-only artifact deposits. Claude Code and Codex can consume Claude Directives materialized into their local harnesses, while exposing the same rules and skills through MCP allows MCP-capable local models such as LM Studio models to consult the same policy and skill set. In addition, an LLM can create `sv://artifact/...` objects through `sourcevault_deposit`, allowing SourceVault to act as an artifact-exchange plane among LLMs and workers. This is compatible with the single-committer principle because workers still cannot write directly to a live Notebook.

For autonomous execution, `SourceVault_autotrigger` can monitor schedules, SourceVault events, and conditions, construct jobs, and asynchronously launch workflows and CatalogWorkflows. The design specification proposes that the first automatically launched workflow should be a comprehensive health check based on `SourceVaultSystemDoctor` and diagnostics; ordinary workflows should run only when required components are healthy or are in an explicitly tolerated degraded state.

In the current implementation, a lightweight-doctor dispatch gate, machine heartbeats, specific-machine dispatch, and remote-result publication are implemented, while automatic triggering of the comprehensive doctor and full WorkerPool-based automatic load balancing/failover remain incomplete.

When multiple PCs share the same SourceVault, per-machine single-writer job/run logs, machine heartbeats, aggregator rollups, `specificMachine` placement, atomic dispatch claims, headless dispatch, and license-slot guards already make it possible to execute workflows remotely on a designated PC and share the result through the vault. These mechanisms form a foundation for redundancy and load balancing, but health-aware WorkerPool selection and complete failover are

still partly design-stage features, so implemented capabilities and future architecture must be evaluated separately.

Taken together, the four primary repositories and the supporting `claudecode` layer are not merely a set of tools for calling LLMs from Notebooks.

The system is evolving into a layered operating model for Wolfram Notebooks that connects knowledge acquisition, URI-based referencing, structuring, search, mining, LLM reasoning, workflow generation, autonomous triggering, multi-PC execution, and auditing while preserving privacy and capability boundaries.

```
Principal code and documentation sources: SourceVault_info/docs/user_manual.md /
SourceVault_info/docs/api_promptrouter.md / SourceVault_info/docs/api_mcp.md /
SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md /
SourceVault_info/design/sourcevault_auto_trigger_scheduler_spec_v0_1.md /
claudecode_info/docs/api.md
```

Chapter 1. Scope of the Study and Overall System Architecture

NBAccess, ClaudeRuntime, ClaudeOrchestrator, and SourceVault are easier to understand as a single layered system than as four independent tools. Beginning with the Notebook, the layers divide responsibility for LLM reasoning, workflow execution, knowledge persistence, and auditing. The key point is that the repositories do not redundantly implement the same functions: responsibilities closer to dangerous side effects and confidential information are placed lower in the stack, while higher layers focus more on planning, orchestration, and knowledge use.

Understanding the current operational environment also requires looking beyond the four repositories and treating `claudecode` as a background integration and compatibility layer. Core functions accumulated historically in `claudecode` - ClaudeEval, provider selection, Claude Directives, the shared polling tick, the palette, and the MCP server registry - still bind the four layers together. It is therefore useful to distinguish between the four-layer logical architecture and `claudecode` as transitional implementation infrastructure.

Usage has also expanded well beyond one-shot ClaudeEval interactions. With deterministic PromptRouter routes, workflow generation for complex tasks through PromptWorkflow, requirements-to-specification and specification-to-implementation pipelines through `spec-review/spec-impl`, sharing with external LLMs through MCP, and autonomous startup through AutoTrigger, the system has grown from a Notebook-operation tool into a knowledge-and-execution platform.

1.1 Repositories Covered

The study covers the following four repositories:

- `transreal/NBAccess`

- transreal/ClaudeRuntime
- transreal/ClaudeOrchestrator
- transreal/SourceVault

At the time of the original survey, each README stated the MIT License and each repository used `main` as its default branch.

Their scale and character differ substantially. NAccess is a relatively small foundational package. ClaudeRuntime combines a single-agent execution kernel with external execution, process supervision, seat management, task placement, and related mechanisms. ClaudeOrchestrator is a higher-level control layer containing workflows, observability, PromptWorkflow, and session handling. SourceVault is the largest repository and contains many subpackages covering source management, search, web, mail, identity, mining, service management, and more.

1.2 Four-Layer Architecture

The basic dependency structure can be represented as shown in Figure 1.

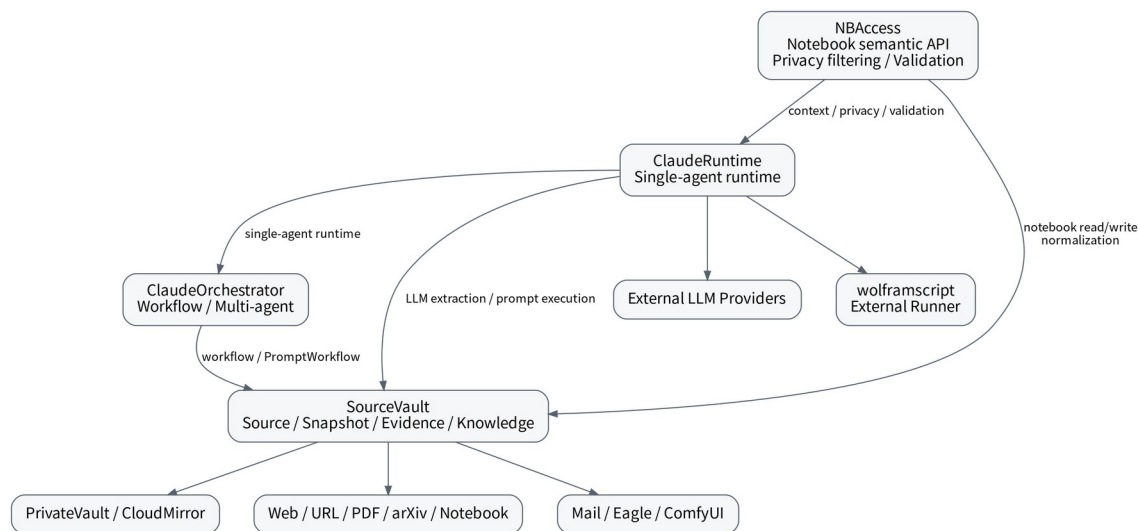


Figure 1. Four-layer architecture and its relationship with external systems

An important point in this diagram is that ClaudeRuntime does not itself own Notebook or secret semantics. Instead, it calls NAccess through an adapter. SourceVault manages the source lifecycle, but delegates safe expression execution to NAccess and multi-worker coordination to ClaudeOrchestrator.

The actual system is not a simple one-way stack; it has the cyclical structure shown in Figure 2.

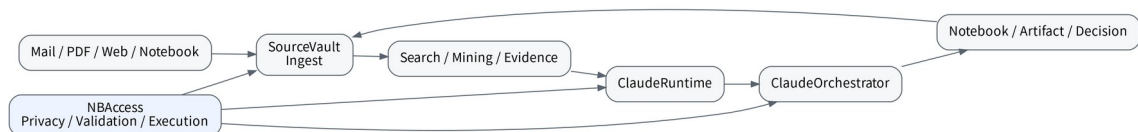


Figure 2. Knowledge-and-execution cycle centered on SourceVault

Evidence retrieved from SourceVault is passed to Runtime/Orchestrator, and generated Notebooks, code, reports, decisions, and workflow logs are written back to SourceVault. As a result, previous work products become inputs to subsequent exploration and reasoning.

1.3 Principal Intended Use Cases

The intended uses of the four repositories can be divided broadly into four categories.

The first is to operate confidential Notebooks while safely routing information between local and cloud LLMs.

The second is to decompose complex tasks into multiple subtasks, execute workers either in parallel or according to dependencies, verify the resulting artifacts, and commit them into a single Notebook.

The third is to preserve URLs, PDFs, arXiv papers, Notebooks, mail, and other material as immutable snapshots and manage the relationships among claims, evidence, and derived artifacts.

The fourth is to transform frequently used natural-language instructions into deterministic function calls or unevaluated expressions so that semantically identical operations can be reproduced.

Even at this level, the design is more tightly optimized for Notebook-centered use than typical RAG or agent frameworks. As discussed later, the mining and safety layers further strengthen its character as a long-term knowledge platform.

1.4 `claudecode` - the Less Visible Integration and Compatibility Layer

Although the four repositories are the primary subjects of this report, an accurate understanding of the current codebase requires considering `transreal/claudecode`. It provides an integrated interface from Wolfram Language/Mathematica to the Claude Code CLI, Codex CLI, Anthropic/OpenAI APIs, LM Studio, and other providers, and still contains many basic functions such as `ClaudeEval`, provider/model selection, Notebook-side UI, the shared polling tick, conditional injection of external package API documentation, and an MCP server registry for CLI clients.

Representative function on the <code>claudecode</code> side	Relationship to the four-layer stack
<code>ClaudeEval / ContinueEval</code>	Notebook-side entry point into Runtime proposal loops and <code>PromptRouter</code>
<code>\$ClaudeModel / \$ClaudeAdvisaryModel / \$ClaudePrivateModel</code>	Selection of normal, verifier/adversarial, and private models
<code>Claude Directives</code>	Projection of rules and skills to Claude Code, Codex, and workers
<code>ClaudeRegisterPollingTick</code> family	Shared tick infrastructure for diagnostics, <code>AutoTrigger</code> , and background-job monitoring
<code>\$ClaudeCLIMCPServers</code>	Neutral registry for registering SourceVault MCP and other servers into headless CLI harnesses
<code>Palette / Front End</code> integration	Existing UI glue for specification generation,

Representative function on the <code>claudecode</code> side	Relationship to the four-layer stack
	implementation, job progress, and result write-back

Thus, while the logical architecture treats the responsibility boundaries of NBAccess, Runtime, Orchestrator, and SourceVault as canonical, the current transitional implementation is best understood as using `claudecode` as a background glue layer for bootstrap, UI, polling, and provider compatibility.

A natural future direction would be to split the enlarged `claudecode.wl` into a core layer and a legacy compatibility layer and move toward an operating model centered on ClaudeOrchestrator, ClaudeRuntime, `claudecode_core`, and NBAccess. However, as of July 16, 2026, no completed `claudecode_core` split could be confirmed in the public repositories; this should therefore be treated as an architectural outlook rather than a current fact.

1.5 Expansion from “Question and Answer” to Knowledge, Workflow, and Service

From the user’s perspective, the four-layer stack now has at least four kinds of entry points. The first is interactive expression proposal through ClaudeEval. The second is deterministic high-level API invocation through PromptRouter. The third is long-running, multi-stage processing through ClaudeOrchestrator/SourceVault workflows. The fourth is service-like autonomous processing, including periods when no Notebook is open, through MCP and AutoTrigger.

Entry point	Typical example	Execution path
ClaudeEval	“Summarize this Notebook,” “Show my schedule”	<code>claudecode</code> -> Runtime -> NBAccess
PromptRouter	Saved routes, UpcomingSchedule, FindNotebooks	SourceVault route -> held expression
Workflow	Requirements -> spec, spec -> implementation, research -> verification -> commit	Orchestrator / SourceVault workflow
MCP	Search from LM Studio, consult Directives, deposit artifacts	MCP service kernel + adapters
AutoTrigger	Launch workflows on time/event conditions, dispatch to a specific PC	SourceVault scheduler + executor

Principal code and documentation sources: `claudecode_info/docs/api.md` / `SourceVault_info/docs/user_manual.md` / `SourceVault_info/docs/api_workflowregistry.md`

1.6 Outlook

A major architectural focus going forward is to make responsibility boundaries even more explicit and to clean up transitional glue code. In particular, separating the functions still residing in `claudecode.wl` into a permanent core and a legacy compatibility layer, and fixing the boundaries among ClaudeRuntime, ClaudeOrchestrator, NBAccess, and SourceVault as stable API contracts, would make the overall system easier to understand and maintain.

Another focus is to make the four layers observable as a single operating model rather than merely as a collection of libraries. If a request could be traced end-to-end - from its entry point, through privacy decisions, LLM and workflow execution, to the `sv://` objects or Notebook cells receiving the result - the architectural separation of responsibilities would also be reflected in operational observability.

In the long term, the natural direction is for the Notebook-centered interactive environment, headless services, MCP clients, and multi-PC workers to share the same policy, identity, and provenance model. Preserving the rule that higher layers may never expand the authority of lower layers will remain central to the system's distinctive architecture. # Chapter 2. NBAccess

NBAccess is the final boundary at the bottom of the four-layer stack, responsible for Notebook operations, confidentiality, access control, and safe expression execution. No matter how sophisticated the plans produced by Runtime or Orchestrator may be, NBAccess decides how much confidential Notebook information may be exposed, whether a proposed Wolfram expression may actually be executed, and how secret keys are handled. Understanding NBAccess therefore explains why the higher layers are intentionally prevented from operating on Notebooks without constraint.

A particularly important principle is that privacy does not apply only to the “secret cell itself”; it propagates to variables and outputs derived from that cell. Even if a value is not directly disclosed, a result computed from a secret value may still leak information. For this reason, dependencies are analyzed before Notebook context is passed to an LLM, and the system decides whether to provide full content or only a schema-level projection. This idea also carries forward into SourceVault privacy propagation and MCP release gates.

Another core principle is that LLM output is treated as a Held Expression rather than as a command. Proposed expressions are inspected before evaluation by functions such as `NBValidateHeldExpr` and classified into decisions such as `Permit`, `NeedsApproval`, `Deny`, or `RepairNeeded`. This also serves as a defense against prompt injection and erroneous generation by structurally eliminating the path in which an expression is executed merely because “the LLM said so.”

2.1 Role

NBAccess is a foundational layer that treats a Mathematica Notebook as an array of cells and provides cell-level reading and writing, privacy evaluation, dependency analysis, history management, and safe expression execution.

Its central characteristic is that when Notebook information is supplied to an LLM, it does not simply stringify the entire Notebook. Instead, it evaluates each cell's privacy level and its dependency relationships.

In the default privacy model described in the README, values are represented approximately on a scale from 0.0 to 1.0, and `AccessLevel -> 0.5` is treated as the boundary for information that may be sent to the cloud. Cells with high privacy, and outputs generated from them through dependency chains, are excluded from cloud-facing context.

2.2 Principal Responsibilities

Responsibility	Description
Notebook semantic access	Unified manipulation of cell retrieval, position, type, style, and content
Privacy filtering	Selection based on per-cell <code>PrivacyLevel</code> and <code>AccessLevel</code>
Dependency propagation	Propagation of confidentiality to outputs that depend on secret variables
Context projection	Generation of task-specific context such as full content, summaries, or schema-only views
Path/value normalization	Generation of SourceVault-oriented object specifications
History	Notebook change history and diffs
Safe execution	Validation, approval, and execution of held expressions
Crypto	Encryption, MAC operations, and key isolation through <code>KeyRef</code>

The principal files are as follows.

File	Primary role
<code>NBAccess.wl</code>	Notebook operations, privacy, dependency analysis, safe execution
<code>NBAccess_crypto.wl</code>	<code>KeyRef</code> -based encryption, MAC, and key management
<code>NBAccess_info/</code>	Supplemental design and API documentation

2.3 Principal APIs

API	Role
<code>NBGetContext</code>	Obtain Notebook context after applying privacy

API	Role
	filters
NBBuildGlobalVarDependencies	Build variable dependency relationships
NBExtractAssignments	Extract assignment expressions
NBFileSpec	Convert a file into a SourceVault-oriented object specification
NBNormalizePath	Normalize paths
NBValueSpec	Convert structural information about a value into an object specification
NBValidateHeldExpr	Decide whether a held expression is safe
API	Role
NBExecuteHeldExpr	Execute an approved held expression
NBAuthorize	Make policy-based execution authorization decisions
NBGenerateSymmetricKeyRef	Generate a symmetric key and return a KeyRef
NBEncryptWithKeyRef	Encrypt data using a KeyRef
NBVerifyMacWithKeyRef	Verify a MAC

2.4 Privacy and Dependency Propagation

Protecting confidential information requires more than simply excluding secret cells.

```

Secret Cell
  ↓
secretVariable
  ↓
derivedOutput

```

Even if `derivedOutput` does not literally contain the secret value, the output may still reveal information if it was computed from `secretVariable`.

NBAccess analyzes assignments, variable references, and output dependencies within the Notebook and propagates confidentiality accordingly. It then constructs context appropriate to the destination and intended use.

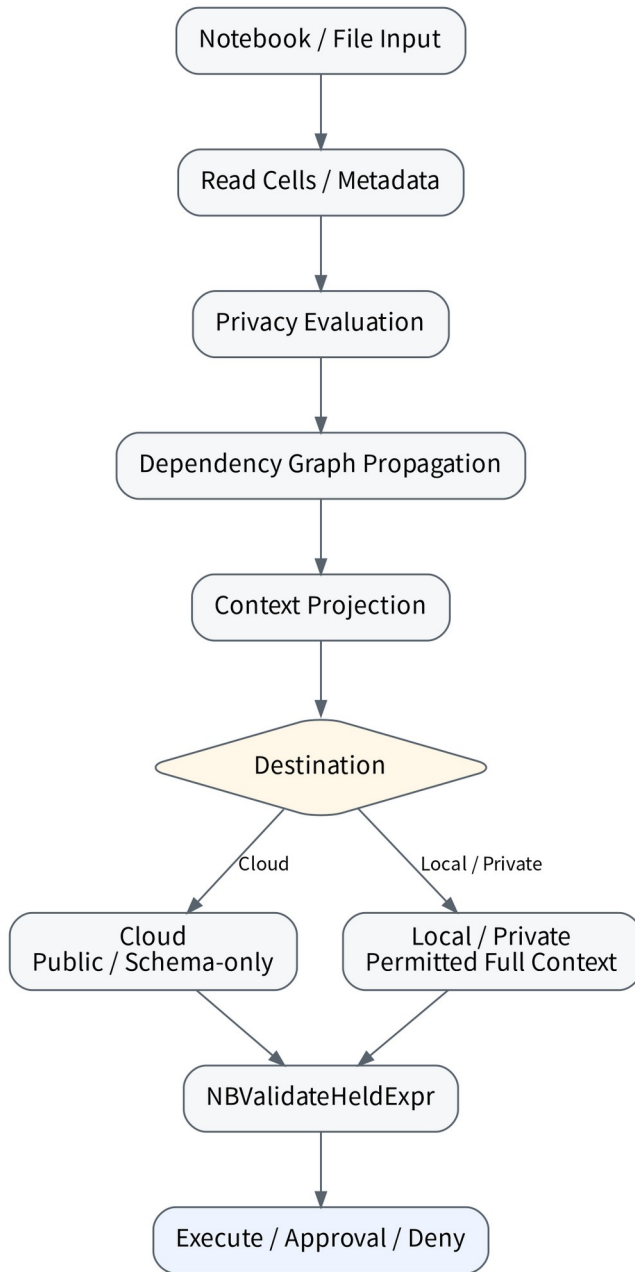


Figure 3. Privacy evaluation, context projection, and safe execution in NBAccess

For outputs that depend on secrets, the system can omit the body and provide only a schema describing aspects such as type, size, keys, and element count. This lets the LLM understand the computational structure without exposing the underlying values.

2.5 Pseudocode for File Privacy

The behavior around `NBFileSpec` can be represented conceptually as follows.

```

function NBFileSpec(path):
    if file does not exist:
        return Exists=False, PrivacyLevel=1.0

    ext = file extension

    if ext == ".nb":
        open notebook safely
        evaluate cell privacy
        count public/confidential cells
        propagate confidentiality through dependencies

        if notebook declares CloudPublishable=True:
            apply controlled public boundary
        if notebook declares CloudPublishable=False:
            PrivacyLevel = 1.0
    else:
        PrivacyLevel = file privacy policy result

    calculate content hash
    construct normalized path/value references
    cache base result
    reapply declared boundary after cache lookup
    return FileSpec

```

A particularly important detail is that Notebook-level declarations are re-applied after crossing the cache boundary, so that a stale cached privacy decision cannot hide the Notebook's latest declaration.

2.6 Safe Expression Execution

The design does not evaluate a Wolfram Language expression returned by an LLM directly through `ToExpression` or an equivalent path.

LLM Output

↓

`HoldComplete[expression]`

↓

`NBValidateHeldExpr`

↓

Permit / NeedsApproval / Deny / RepairNeeded

↓

`NBExecuteHeldExpr`

Dangerous heads, Notebook writes, external processes, credentials, network access, and similar operations are classified according to policy.

Because NBAccess owns this final decision, permissions cannot be expanded merely by changing Runtime- or Orchestrator-side settings.

2.7 Cryptographic Layer

NBAccess_crypto.wl exposes KeyRef values rather than raw keys through its external API.

Conceptually, it operates as follows.

```
function NBEncryptWithKeyRef(keyRef, plaintext):
    key = resolve internally
    if key type is not symmetric:
        fail

    encrypted = Encrypt(key, plaintext)

    if encryption failed:
        fail

    return serialized encrypted object
```

MAC verification uses constant-time comparison rather than ordinary string comparison.

```
function NBVerifyMacWithKeyRef(keyRef, bytes, expectedMac):
    actualMac = calculate MAC using internal key
    return constant_time_equal(actualMac, expectedMac)
```

Key material is not returned through normal APIs or written to ordinary logs, and the keyRef index stores metadata only. A SystemCredential-based configuration is recommended; the in-memory backend is intended mainly for development and testing.

2.8 Assessment

NBAccess is the clearest security boundary among the four repositories.

Its strength lies in designing Notebook cells, variable dependencies, held expressions, safe execution, and cryptographic keys as parts of one security boundary.

At the same time, defects in this layer can affect the entire higher-level stack. It is therefore also the package that most strongly requires rigorous regression tests, property tests, boundary-value tests, and cross-kernel tests.

2.9 Outlook

The highest priority for NBAccess is the formalization and regression testing of policy boundaries. In particular, confidential dependency propagation, CloudPublishable boundaries, resistance to held-expression bypasses, and the atomicity of Notebook semantic writes are worth strengthening with more property tests and cross-kernel tests because they underpin the security of the entire upper stack.

The current Notebook-centered privacy model also needs to be applied consistently to `sv://` objects, MCP projections, and remote workflow results. NBAccess does not need to understand every medium directly, but it would be advantageous if the same policy snapshot could be referenced at the boundaries of final Notebook writes, context releases that may contain secrets, and external side effects. This would align NBAccess more strongly with SourceVault release decisions.

In the longer term, it would be desirable to express NBAccess policy declaratively as an `AccessSpec` and have Runtime, Orchestrator, and MCP all refer to the same digest. This would further reduce TOCTOU problems in which policy changes mid-execution, as well as policy drift across processes. # Chapter 3. ClaudeRuntime

ClaudeRuntime is the layer that manages one execution cycle of “propose an expression from a prompt, validate it, and then execute it.” NBAccess owns the final safety decision; Runtime manages the conversational state with the LLM, provider calls, proposal parsing, repair, and continuation decisions. Runtime is therefore not the source of truth for safety. It is a control engine that carries proposals toward the safety boundary.

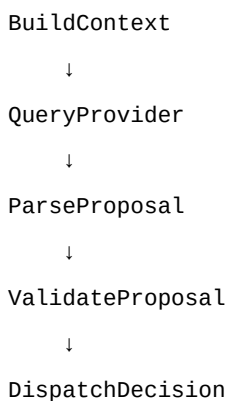
It is important to understand that what appears externally as a single `ClaudeEval` call actually contains the stages `BuildContext`, `QueryProvider`, `ParseProposal`, `ValidateProposal`, and `DispatchDecision`. Whether `PromptRouter` returns a deterministic unevaluated expression or an ordinary LLM proposes an expression, converging both cases on the same proposal contract reduces differences among entry paths.

Runtime also includes external runners, a process supervisor, a seat broker, and task placement. These mechanisms allow heavy processing to be separated from the main kernel so that long-running work does not block the Notebook Front End. At the same time, Runtime must therefore handle operational failures that are fundamentally different from LLM errors, such as PID reuse, license-seat exhaustion, and cleanup failures.

3.1 Role

ClaudeRuntime is an Expression-Proposal loop state machine that manages one or more continuing turns of a single agent.

The basic flow has five stages.



Runtime itself does not own the semantics of Notebooks, secrets, or access policy. Those are delegated to adapters and NBAccess. Through this separation of responsibilities, Runtime is responsible not for deciding “what may be executed,” but for deciding “in what sequence a proposal is processed and to which decision state it proceeds.”

3.2 Principal Files

File	Primary role
ClaudeRuntime.wl	RuntimeState, turn DAG, proposal loop
ClaudeRuntime_externalrunner.wl	External execution through wolframscript
ClaudeRuntime_processsupervisor.wl	External process supervision
ClaudeRuntime_seatbroker.wl	Execution-seat management
ClaudeRuntime_session.wl	Runtime sessions
ClaudeRuntime_sessionrunner.wl	Session execution support
ClaudeRuntime_task_placement.wl	Task classification and backend recommendations
ClaudeRuntime_info/	Design and API documentation

3.3 Adapter Model

When Runtime is constructed, the adapter provides the required functions.

BuildContext
QueryProvider
ParseProposal
ValidateProposal
ExecuteProposal
RedactResult
ShouldContinue

This adapter structure separates LLM providers, CLIs, local models, Notebook context, and execution methods from the Runtime core.

3.4 Turn Execution

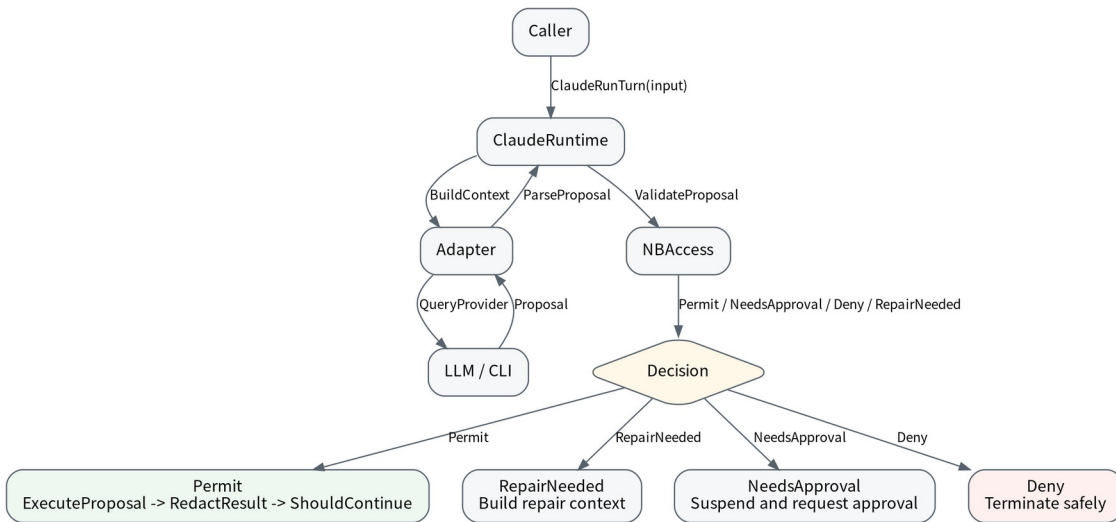


Figure 4. ClaudeRuntime Expression-Proposal loop

ClaudeRunTurn constructs nodes such as buildContext, queryProvider, parseProposal, validateProposal, and dispatchDecision as a DAG and launches the job.

The central design choice is that an LLM response is not evaluated directly; it is parsed as a proposal and must pass NBAccess validation.

3.5 Decision Model

Decision	Meaning
Permit	Validated and executable
NeedsApproval	Requires user approval
Deny	Not executable
RepairNeeded	Proposal needs repair
TextOnly	Text response rather than expression execution
ToolUse	Continue through an explicit tool path

This classification means that responses that cannot be executed are not all treated as generic errors; they can branch into text responses, repair, approval waiting, and other states.

3.6 Task Placement

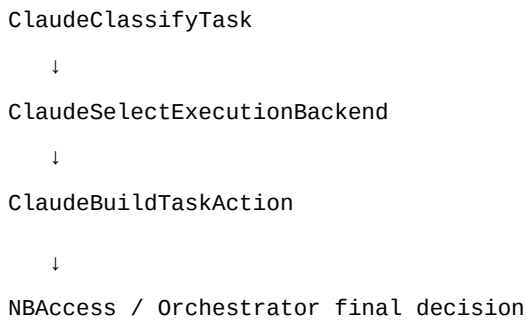
ClaudeRuntime_task_placement.wl provides advisory logic for deciding where a task should run.

Raw Task

↓

ClaudeNormalizeTaskSpec

↓



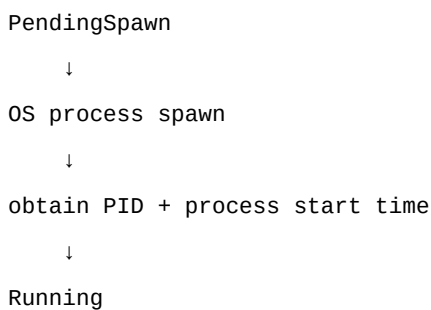
Representative execution backends can be organized as follows.

Execution backend	Appropriate processing
Main Kernel	Notebook UI, Front End dependencies, live-state operations
Subkernel	Side-effect-free numerical computation and parallel pure computation
WolframScript	Long-running batches, external I/O, large-scale ingest, independent processes

Final authorization is determined by NAccess policy and Orchestrator workflow policy. Runtime’s recommendation alone cannot move a task to a dangerous backend.

3.7 External Process Management

External runners separate long-running processing from the main kernel. Process Supervisor uses a two-stage manifest.



Recording the process start time in addition to the PID reduces the risk of accidentally terminating a different process after the operating system reuses a PID.

On termination, seat release, archiving, and cleanup are sequenced, and cleanup failures are retained as diagnostic events.

3.8 Assessment

ClaudeRuntime’s greatest strength is its clear single responsibility. By delegating safety decisions to NAccess, multi-agent orchestration to Orchestrator, and long-term knowledge to SourceVault, the Runtime core can focus on the proposal loop.

At the same time, the relationships among adapters, legacy `claudecode`, external runners, sessions, and task placement are complex. Load order, compatibility modes, and fallback paths therefore need to be managed explicitly.

3.9 ClaudeEval Is Not the Only Entry Point into Runtime

A point to keep in mind when understanding `ClaudeRuntime` is that `ClaudeEval` is not necessarily a function that always asks an LLM to generate freely. At the current Notebook-side entry point, a saved `PromptRoute` or a deterministic `SourceVault` route can be proposed before any LLM call is made.

`SourceVaultProposePromptRoute` returns an unevaluated expression wrapped in `HoldComplete[...]`, not an evaluated result. Only that expression is passed to `Runtime`, where it undergoes a head allowlist check and `NBAccess` validation before execution.

For example, a routine request such as “Show my schedule for the next three days” can be transformed into a frozen function call to `SourceVaultUpcomingSchedule` instead of being reinterpreted by a heavyweight LLM every time. When a saved prompt matches exactly, `SourceVaultProposeSavedPromptRoute` can also present an existing route before an LLM is called. This enables low-cost, reproducible re-execution while preserving `ClaudeEval`’s contract of pre-evaluation validation.

```
ClaudeEval[prompt]
  ↓
saved route / FunctionRoute / lexical route
  ↓
unevaluated HoldComplete[SourceVaultFunction[...]]
  ↓
Runtime head check + NBAccess validation
  ↓
execution
if unresolved
  ↓
single-step Heavy LLM or PromptWorkflow candidate
```

3.10 Responsibility Boundary Between Runtime and PromptRouter

`PromptRouter` determines which high-level function a request should be routed to, but it is not an authority that grants execution rights. `Runtime`, on the other hand, manages turn progression, including provider interaction, proposal parsing, repair, and approval, but does not own Notebook privacy or final execution authorization. Through this separation, both deterministic expressions created by `PromptRouter` and freely generated expressions created by an LLM ultimately converge on the same held-expression validation boundary.

Principal code and documentation sources: `SourceVault_info/docs/api_promptrouter.md` / `SourceVault_info/docs/user_manual.md` / `claudecode_info/docs/api.md`

3.11 Outlook

A key future task for Runtime is to formalize `RuntimeSession` as a public lifecycle rather than treating only individual turns as first-class. The current code already has `ConversationState`, `TurnCount`, tool loops, and checkpoint material, but durable checkpoints across process restarts, budget suspend/resume, and per-tool-call approval have not yet been unified into a complete session protocol.

From the perspective of prompt-injection defense, another important step is to make `PreparedInputToken` verification fully enforced at the Runtime/adaptor boundary immediately before LLM transmission. Fixing the entire request envelope - provider, messages, tool schemas, and retrieval references - and verifying it just before sending would prevent substitutions after context construction.

Task placement also has room to evolve from performance optimization into a broader execution contract incorporating privacy, license seats, GPU/CPU resources, and `SpecificMachine` constraints. Connecting this to Orchestrator's resource placement would naturally extend single-turn execution placement into multi-PC resource scheduling. # Chapter 4. ClaudeOrchestrator

ClaudeOrchestrator is the layer that decomposes complex requests across multiple workers, integrates their artifacts, and carries the result through to final commit. The system is easiest to understand as the sequence Planning -> Spawn -> Reduce -> Commit. The planner decomposes work into `TaskSpecS`, workers produce individual artifacts, the reducer integrates those artifacts, and only the single committer is allowed to apply the final result to live state such as a Notebook.

This single-committer principle is not merely an implementation convenience; it is central to safety and reproducibility. If multiple workers write directly into the same Notebook, the system becomes vulnerable to conflicts, ordering dependencies, partial failures, and difficult rollback. Workers are therefore restricted to artifact production, while shared outputs are passed through SourceVault or explicit artifact references. MCP deposits and `sv://artifact/...` objects are natural media for sharing results among agents while preserving this principle.

Orchestrator also uses Petri-net workflows, allowing it to represent states that are awkward to model with a simple DAG: waiting for approval, retry loops, joins across branches, waiting for an external job, cancellation, and recovery. `PromptWorkflow` can transform complex natural-language requests into workflow candidates, while `spec-review` and `spec-impl` make the software-development process itself - generating a specification from requirements and then implementing and verifying it - a workflow.

4.1 Role

ClaudeOrchestrator is the multi-agent orchestration layer that coordinates multiple Runtime instances or workers from above.

Its basic flow consists of four stages.

Planning

↓

Spawn

↓

Reduce

↓

Commit

Where `ClaudeRuntime` manages one proposal loop, `ClaudeOrchestrator` manages task decomposition, dependencies, workers, artifacts, and commit ordering.

4.2 Core Architectural Shift

Earlier designs in which workers wrote directly to live Notebooks had a number of problems: inconsistent shared state across multiple workers, unstable resolution of `EvaluationNotebook[]`, unintended execution of `CreateNotebook`, nondeterministic commit ordering, and difficult rollback.

The design therefore shifted to one in which workers are limited to artifact production and only a single committer may modify the live Notebook.

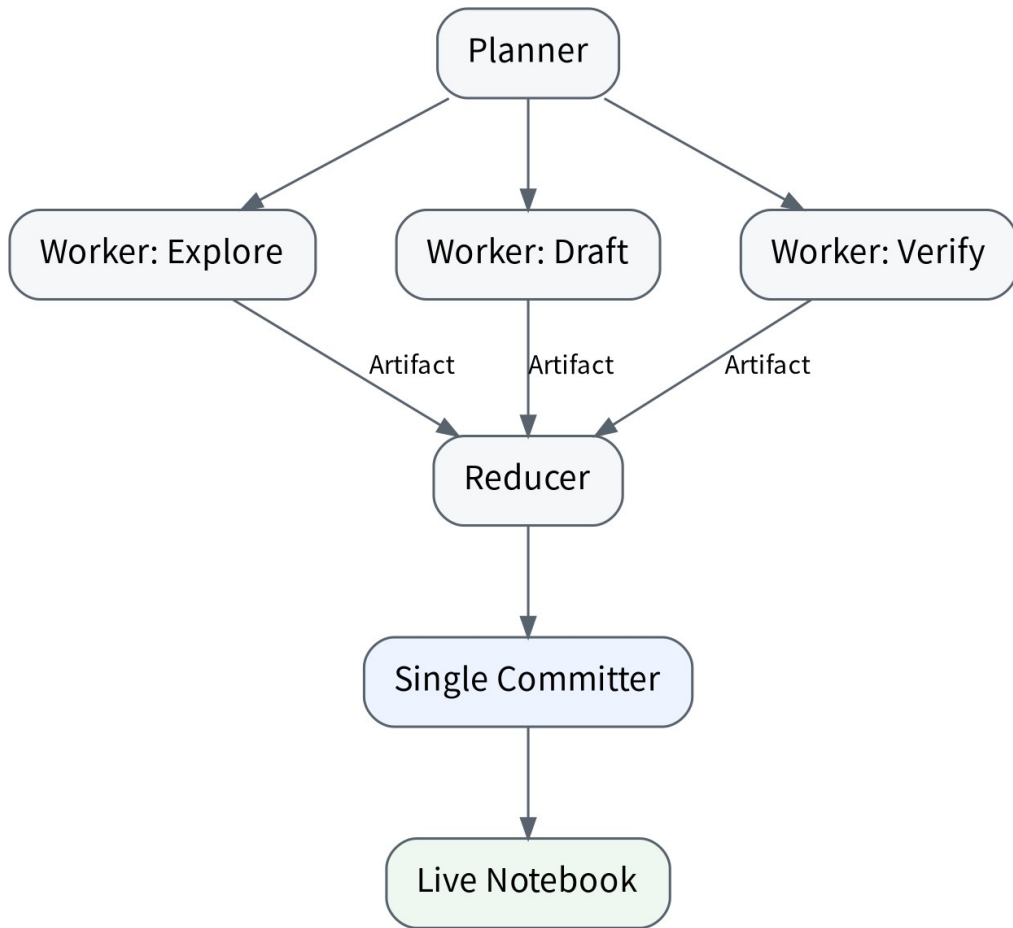


Figure 5. ClaudeOrchestrator worker/reducer/single-committer structure

What workers share is not the Notebook itself, but artifacts, JSON, Associations, and explicit references.

4.3 Principal Files

File	Primary role
ClaudeOrchestrator.wl	Roles, capabilities, planner, spawn, reduce, commit
ClaudeOrchestrator_workflow.wl	Multi-token Petri net
ClaudeOrchestrator_promptworkflow.wl	Conversion of complex prompts into workflows
ClaudeOrchestrator_observability.wl	Observability of LLM calls, handlers, and workflows
ClaudeOrchestrator_session.wl	Session support
ClaudeOrchestrator_info/	Design, API, and examples

4.4 Roles and Capabilities

Representative roles include Explore, Plan, Draft, Verify, Reduce, and Commit.

Each role is assigned capabilities, and worker roles are not granted direct write authority over Notebooks.

Representative denied operations include `NotebookWrite`, `CreateNotebook`, `EvaluationNotebook`, `RunProcess`, `SystemCredential`, unrestricted `URLRead`, and direct side effects on live resources.

4.5 Planning

`ClaudePlanTasks` decomposes a parent task into a `TaskSpec` DAG.

`TaskId`

`Role`

`Goal`

`Inputs`

`Outputs`

`Capabilities`

`DependsOn`

`ExpectedArtifactType`

`OutputSchema`

Planner output is not executed directly; its schema and dependencies are validated first.

4.6 Spawn

`ClaudeSpawnWorkers` launches workers while respecting dependencies and collects their artifacts.

Task A: Explore

Task B: Draft, depends on A

Task C: Verify, depends on B

Each worker receives only its own `TaskSpec`, permitted inputs, capabilities, and output schema.

Results are aggregated into `Artifacts`: `taskId` -> artifact and `Failures`.

4.7 Reduce

`ClaudeReduceArtifacts` integrates multiple artifacts. Reduction may use deterministic processing or LLM-based synthesis, but source references, task references, and verification state must be preserved.

The reducer does not write to the live Notebook; it produces a `ReducedArtifact`.

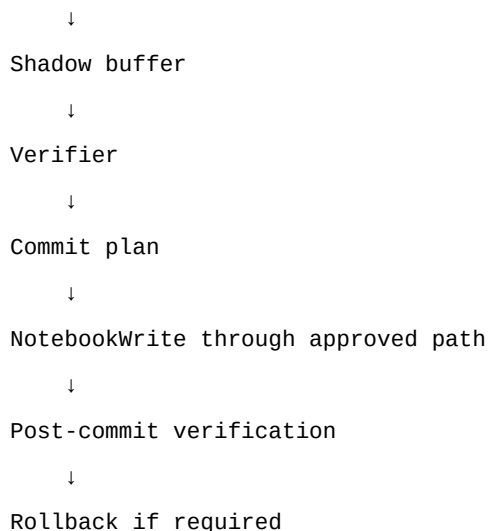
4.8 Commit

`ClaudeCommitArtifacts` performs final application as the single committer.

`Reduced Artifact`

↓

Schema validation



This prevents an erroneous worker output from directly damaging the Notebook.

4.9 Petri-Net Workflows

`ClaudeOrchestrator_workflow.wl` provides a multi-token Petri net that is more general than a simple DAG. Petri nets are well suited to expressing transitions that fire only when multiple conditions are satisfied, multiple tokens, retry, approval waits, waits for external jobs, branch joins, cancellation, timeout, and repair loops.

4.10 PromptWorkflow

PromptWorkflow converts a complex natural-language prompt into a workflow rather than handling it as a single LLM turn. It also classifies short factual questions and simple function queries back onto Runtime's single-path route.

4.11 Assessment

ClaudeOrchestrator's strongest architectural choice is the separation between workers and the committer. In many multi-agent systems, each agent can invoke tools directly and apply side effects to a shared environment. Here, workers are deliberately limited to artifact production and commit is made an explicit, separate responsibility. This increases Notebook consistency and auditability.

4.12 PromptWorkflow - Converting Complex Natural-Language Requests into Workflow Candidates

`ClaudeOrchestrator_promptworkflow.wl` handles prompts that are difficult to represent as a single function route and converts them into WorkflowNet candidates. Even here, a newly generated workflow is not executed the moment an LLM emits it. If an existing `FunctionRoute` or `WorkflowRoute` matches uniquely, that route is preferred and new generation is avoided. If no existing route applies, a local complexity detector runs before any LLM call. An explicit request to "make this a workflow," the presence of multiple subtask verbs, or a combination of task verbs with control words such as "first," "next," or "finally" causes the prompt to be classified as a `WorkflowCandidate`.

When a prompt becomes a candidate, `ClaudeProposeWorkflowNetFromPrompt` proposes `WorkflowNet` code, but the result then passes through static forbidden-pattern scanning, held parsing, and extraction of the declarative structure. A newly generated workflow stops in `NeedsApproval` as a `WorkflowRouteDraft` even if the prompt itself says “create it and run it immediately”; it is not automatically registered or executed. This is an important boundary separating “the LLM’s ability to generate code” from “the authority to activate a workflow.”

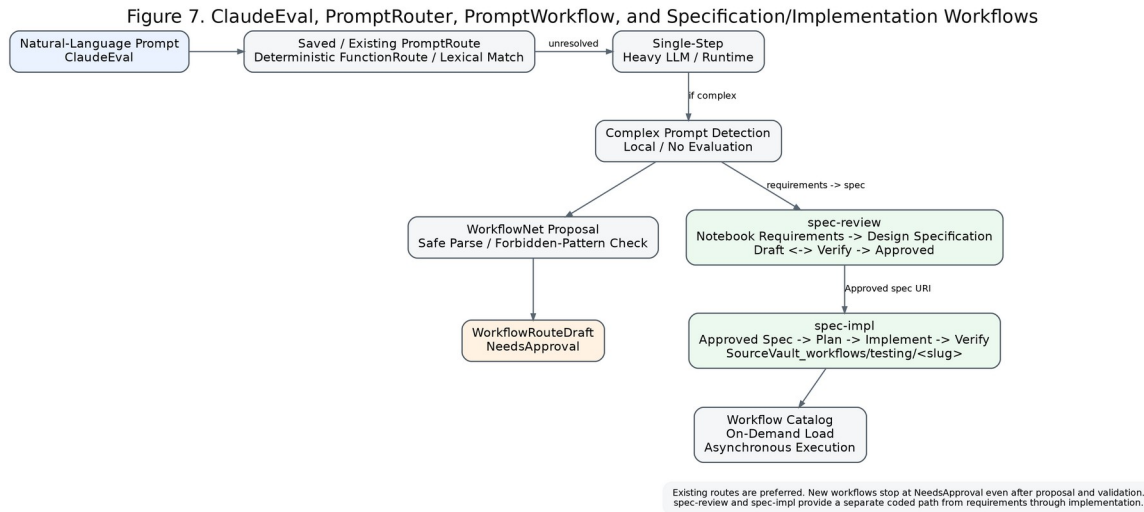


Figure 7. Relationship among *ClaudeEval*, *PromptRouter*, *PromptWorkflow*, and *specification/implementation workflows*

4.13 Turning Requirements-to-Specification and Specification-to-Implementation into Workflows

In addition to `PromptWorkflow`, `SourceVault_workflows` contains two system workflows that automate the software-development process itself: `spec-review` and `spec-impl`. `spec-review` takes requirement text from a Notebook and iterates between a drafting model (`$ClaudeModel`) and a verifier model (`$ClaudeAdvisaryModel`, with Codex as the default) until a Wolfram design specification in Markdown reaches `Approved` status.

The approved specification and its review are stored as immutable `SourceVault` snapshots, and the approved specification can be referenced as `sv://snapshot/OrchSpec/...`

`spec-impl` takes an approved specification and iterates `Plan -> optional plan review -> Implement -> Verify`, generating a loadable `svWorkflow_Canon` package under

`SourceVault_workflows/testing/<slug>/. Verification does not depend only on an LLM review. It includes multiple stages: static smoke tests, a hard gate that dynamically loads and launches the workflow in a fresh wolframscript, execution of generated tests, and finally LLM-based specification-compliance verification. Newly generated artifacts always enter the testing stage and are promoted to production only after testing.`

Stage	Input	Primary processing	Artifacts in SourceVault
spec-review	Notebook requirements	Draft <-> verify until Approved	orch/<project>/ {requirements, spec, review}; sv://snapshot/OrchSpec/ ...
spec-impl	Approved design specification	Plan -> Implement -> Verify	impl/<slug>/ {plan, planreview, artifact, verify}
Catalog registration	Verified workflow package	Generate into testing stage; on-demand load	SourceVault_workflows/ testing/<slug>/

Heavy LLM loops run in background wolframscript processes so that they do not freeze the Front End kernel, while the Front End monitors progress and results through the shared polling tick. Even on failure, intermediate plans, artifacts, and verification chains remain in SourceVault. This makes the system more than a code generator: the design and verification process itself can be reconstructed.

Principal code and documentation sources: `ClaudeOrchestrator_promptworkflow.wl / SourceVault_workflows/spec-review/SVWorkflow_SpecReview_info/design/implementation-spec.md / SourceVault_workflows/spec-impl/SVWorkflow_SpecImpl_info/design/implementation-spec.md / SourceVault_info/docs/api_workflowregistry.md`

4.14 Outlook

One important future task is to clarify the boundary between `RuntimeSession` and the Petri net. Expanding every model turn and ordinary tool call into Petri tokens would cause state explosion. A more suitable division is for Orchestrator to manage episode start, approval, budgets, and artifact commits, while Runtime encapsulates the internal turn/tool loop.

For inter-agent sharing, using SourceVault as an artifact bus will become increasingly important. If worker A deposits an `sv://artifact/...` object through MCP, worker B can search for and retrieve it, and the reducer can integrate it while preserving the reference graph. This avoids misusing the Notebook itself as shared memory.

In the long term, the goal is to integrate the workflow catalog, AutoTrigger, and multi-PC placement so that the same workflow can be launched consistently by a user, PromptRouter, a schedule, a SourceVault event, or a remote PC. Even in that future, approval, stage promotion, and diagnostics gates should remain between automatic workflow generation and automatic execution. # Chapter 5. Fundamental Architecture of SourceVault

SourceVault is not merely a document-search database. It is designed as a source-first knowledge vault that tracks “what evidence was used, which version was used, and what was generated from

it.” URLs, PDFs, arXiv papers, Notebooks, and mail are registered as sources; their contents at a particular point in time are preserved as immutable snapshots; and summaries, claims, search chunks, and artifacts are managed separately as derived objects.

The advantage of this separation is that the impact of upstream changes can be traced. Even if a URL remains the same, an updated document becomes a different snapshot, and Evidence Bundles or generated artifacts based on the older snapshot can be treated as candidates for staleness.

SourceVault’s core value therefore lies less in search speed than in source identity, versioning, provenance, and reproducibility.

`sv://` URLs are also important for exchange across system boundaries. Objects, snapshots, artifacts, chunks, directives, and related entities can be represented as stable opaque references and referred to consistently across MCP and workflows. This makes it possible to avoid passing enormous bodies of text directly to an LLM and instead retrieve only the required projection, subject to release gates, when needed.

5.1 Source-First Knowledge Vault

SourceVault is the largest of the four repositories and has the broadest functional scope.

Its fundamental philosophy is to canonicalize the identity, version, retrieval time, content hash, and provenance of the source itself before treating LLM-generated summaries or answers as first-class knowledge.

Sources may include URLs, web pages, PDFs, arXiv papers, Notebooks, text, mail, Eagle-managed materials, generated artifacts, workflow results, and other content.

5.2 Source, Snapshot, and Derived Artifact

SourceVault separates the logical source from snapshots representing its contents at particular points in time.

Source

- └─ Snapshot v1
- └─ Snapshot v2
- └─ Snapshot v3

The contents of the same URL or paper may change over time. `source` represents identity, while `snapshot` represents an immutable version based on a content hash.

Summaries, claims, embeddings, search indexes, and generated code are treated as derived artifacts separate from the original source.

Snapshot

- └─ Parsed Text
- └─ Claims
- └─ Summary

- └ Search Chunks
- └ Evidence Bundle

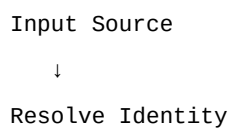
This separation makes it possible to determine which derived artifacts have become outdated when the original material changes.

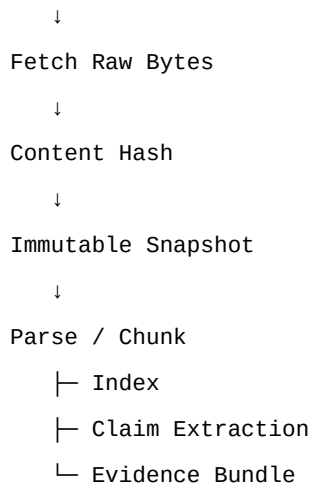
5.3 Principal Files

File	Primary role
SourceVault.wl	Umbrella loader and integrated entry point
SourceVault_core.wl	Immutable snapshots, events, vault core
SourceVault_contracts.wl	FunctionContract, failure schema, initialization DAG
SourceVault_promptrouter.wl	PromptRouter, saved prompts, privacy routing
SourceVault_searchindex.wl	Release context and search index
SourceVault_searchview.wl	Work surface for traversing search results
SourceVault_servicemanager.wl	Services, jobs, heartbeats
SourceVault_webingest.wl	Web/PDF acquisition and analysis
SourceVault_identity.wl	Identifier/Entity model
SourceVault_mining.wl	Tags, authorship, MiningObject

File	Primary role
SourceVault_oopsseed.wl	OOPS seed ontology
SourceVault_mailstructure.wl	General mail structuring
SourceVault_knowledgehome.wl	Knowledge Home
SourceVault_diagnostics.wl	SystemDoctor
SourceVault_capbroker.wl	CapabilityLease / PreparedInputToken
SourceVault_taint.wl	Input trust and taint
SourceVault_cognition.wl	Cognitive assistance
SourceVault_adjudication.wl	Multi-LLM adjudication
SourceVault_anomaly.wl	Observe-only anomaly analysis

5.4 Ingest Flow

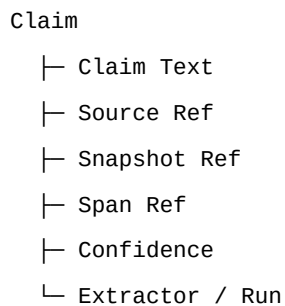




Raw data is stored by content hash, while metadata refers to the source and snapshot. Even when identical content is obtained from different URLs, content hashes can be used to detect and manage duplication.

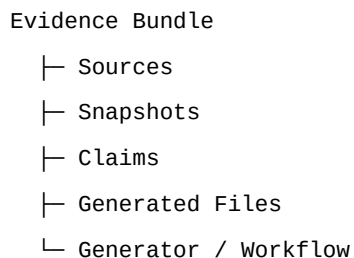
5.5 Claims and Evidence Bundles

A Claim is a proposition extracted from a source.



A claim does not replace the original source. It is managed as an append-only derived record.

An Evidence Bundle groups the sources, snapshots, claims, and generated files on which an artifact depends.



If an upstream snapshot becomes stale or invalidated, bundles that reference it become candidates for re-evaluation.

5.6 PromptRouter

PromptRouter transforms a natural-language prompt into either a deterministic function call or a workflow proposal.

The important point is that the router itself does not evaluate the expression.

Natural Language Prompt

↓

Route Proposal

↓

Held / Unevaluated Expression

↓

NBAccess / Runtime Validation

↓

Execution

If the request is ambiguous, the router does not execute arbitrarily; it returns candidates or NeedsApproval.

5.7 Release Context and Fail-Closed Search

SourceVault search requires a ReleaseContext.

MaxPrivacyLevel

AllowedTags

DenyTags

TrustDomain

Purpose

RevocationEpoch

Search results are re-evaluated by a request-time gate immediately before release, not only at build time.

Search Candidate

↓

Request-time Release Policy

↓

Revocation Check

↓

Safety Check

↓

Permit only

An unregistered or incomplete context does not broaden access. It fails closed, returning an empty result or a refusal.

5.8 FunctionContract

Because SourceVault has a large number of functions, it includes a layer that contractualizes return values, failures, and initialization dependencies. A standardized Failure schema distinguishes among the cause, a publicly releasable reason, and internal diagnostics instead of collapsing everything into `$Failed`.

5.9 Initial Assessment of SourceVault

Even at the time of the original report, SourceVault was more than a document repository.

By integrating immutable snapshots, claims, Evidence Bundles, Notebook management, PromptRouter, and release contexts, it already provided source-centric provenance and deterministic re-execution.

This foundation is what allows the mining layer discussed later to preserve where relationships came from - which snapshot, span, person, topic, or workflow - rather than merely retrieving text that looks similar.

5.10 Practical Use of SourceVault - From Ingest to LLM Use

SourceVault does not begin with the assumption that it is a “vector store for an LLM.” It first registers source identity and version. The basic operations include ingesting URLs/PDFs/text/arXiv materials with `SourceVaultIngest`, indexing Notebooks with `SourceVaultIndexNotebook`, and checking state with `SourceVaultStatus`, `ListSources`, and `ListSnapshots`. Re-ingesting identical content hashes does not create duplicates; the stable identity of a source and its point-in-time snapshots are kept separate.

Usage paths are divided into deterministic paths and LLM paths. Ingest, Notebook indexing, linting, FindNotebooks, and known PromptRoutes can operate without an LLM. Processing that requires semantic interpretation, such as structured claim extraction or Notebook summaries, uses an LLM through ClaudeRuntime. This separation prevents source management itself from depending on a specific LLM provider.

Usage scenario	Representative operation	LLM dependency
Registering materials	<code>SourceVaultIngest[URL/PDF/text/arXiv]</code>	None
Notebook management	<code>SourceVaultIndexNotebook</code> , <code>Header/ToDo/Lint</code>	None
Evidence access	<code>Span/Context</code> , snapshot, Evidence Bundle	None
High-level query	<code>PromptRouter</code> for <code>UpcomingSchedule</code> , etc.	Depends on route; deterministic routes require none
Semantic extraction	<code>SourceVaultExtract</code> via LLM schema, Notebook summary	Through ClaudeRuntime

Usage scenario	Representative operation	LLM dependency
Complex work	Orchestrator workflows, spec-review/spec-impl	May use multiple LLMs
External LLM integration	MCP search/get/directives/deposit	Depends on client/model access profile

5.11 `sv://` URIs - Canonical Identity Across SourceVault Boundaries

The SourceVault Universal MCP Access specification establishes a policy of canonicalizing references exchanged among SourceVault, MCP, Web, and Orchestrator into `sv://...` URIs. An important design choice is not to encode data classes such as mail, web, and PDF directly into the URI namespace. Media type is instead stored as sidecar metadata such as `class`, `MediaType`, and `Kind`, while the URI remains an opaque identity.

Example namespace	Purpose
<code>sv://object/<id></code>	General object identity
<code>sv://chunk/<id></code>	Chunk for search and grounding
<code>sv://artifact/<id></code>	Artifact generated by an LLM or workflow
<code>sv://hash/sha256/<hex></code>	Content-addressed hash
<code>sv://snapshot/<class>/<hex></code>	Immutable snapshot
<code>sv://file/<id></code>	Allow-listed file reference
<code>sv://packageapi/<id></code>	Package API contract/document
<code>sv://directive/<id></code>	Directive identity namespace
<code>sv://group/..., relation, citation, record</code>	Relationships, groups, citations, records

`SourceVaultCanonicalURI` is the normalization point used before storing a URI as a key, edge, group member, or `SourceRef`. Legacy references such as `blob:sha256:...` and `snapshot:class:...` are accepted as compatibility inputs and can be canonicalized into `sv://hash/...` and `sv://snapshot/...`. It would therefore be inaccurate to say that every internal SourceVault record is already stored exclusively as a URI. A more precise description is that `sv://` is becoming the stable common reference layer for external APIs, search, and relationship descriptions. A URI represents identity, not capability; knowing a URI does not itself grant read access.

5.12 Combining LLM Calls with SourceVault

There is no single way to let an LLM use SourceVault. `PromptDeliveryProfile` considers whether the model and execution environment can use MCP, whether they can read local folders, and whether SourceVault inputs are fully mediated, then chooses strategies such as `MCPDeferred`, `InlineContext`, `Hybrid`, `LocalToolRefs`, or `NoSourceVault`. This allows a cloud-backed model with a local tool harness, such as Claude Code, to be distinguished from a truly local LM Studio model, separating provider trust from data release.

In particular, even though Claude Code or Codex can manipulate a local filesystem or shell, the model inference itself is cloud/remote. The MCP release decision must therefore not automatically treat such clients as LocalLLM and release high-privacy bodies. Conversely, a trusted local/private model may receive a higher access ceiling according to its registered AccessProfile.

5.13 SourceVault MCP - External LLM Interface as a Projection Layer

SourceVault_mcp.wl is not the MCP protocol transport itself. It is the Wolfram Language-side library responsible for tool schemas, initialize, tools/list, tools/call dispatch, argument validation, provenance, and JSON-safe responses. HTTP/JSON-RPC transport is handled by a Python proxy, which communicates through a file-command queue to SourceVaultMCPDispatch in the service kernel. MCP is therefore not the authority over the SourceVault data store; it is a safe projection layer. Final decisions over read/write/cloud-send operations are delegated to release policy, AccessGrants, NBAccess, and related authorities.

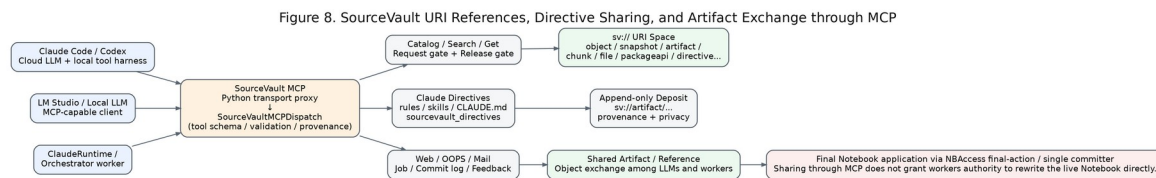


Figure 8. URI references, Directive sharing, and artifact exchange through MCP

5.14 The 26 Tools Provided by the Current MCP Surface

The current SourceVault MCP tool set goes well beyond a simple search API. The following 26 tools are grouped by function.

Category	Tool	Role
Filesystem	sourcevault_fs_list / sourcevault_fs_read	List/read allow-listed package and working directories; reject secret-looking files
Directives	sourcevault_directives	List Claude Directives rules/skills/root instructions and return them as URIs
Workflow authoring	sourcevault_workflow_write	Assist API/LM Studio-style clients in safely writing workflow files under SourceVault_workflows
Orchestrator feedback	sourcevault_orchestrator_help / sourcevault_orchestrator_subtask	Bridge help requests and subtask proposals to Orchestrator
Web	sourcevault_web_search /	Synchronous/asynchronous

Category	Tool	Role
	sourcevault_submit_web_search / sourcevault_job_status / sourcevault_job_result / sourcevault_get_document	web search, jobs, and retrieved documents
Package history	sourcevault_commit_log	Package commit metadata derived from the GitHub API
OOPS	sourcevault_oops_status / sourcevault_oops_search_threads / sourcevault_oops_thread	OOPS corpus status, thread search, and thread retrieval
General mail	sourcevault_mail_status / sourcevault_mail_search_threads / sourcevault_mail_thread	Search structured general-mail data
Universal core	sourcevault_catalog / sourcevault_search / sourcevault_get	Adapter catalog, federated search, single-URI retrieval
Access	sourcevault_request_access / sourcevault_access_status	Request and poll for AccessGrants required for bodies and similar content
Feedback	sourcevault_feedback_submit	Append-only feedback envelopes for critique, correction, help, etc.
Write	sourcevault_deposit	Append-only deposit of LLM-generated artifacts
Capability	sourcevault_runtime_capabilities	Return PromptDeliveryProfile and available capabilities by model/client

sourcevault_search searches across adapters, normalizes a SearchSpec, and passes each result through a request-time release gate.

sourcevault_get resolves an sv:// URI and, by default, returns only low-leakage summary/metadata. Body/raw/context access requires an AccessGrant, and sending bodies to a cloud sink is designed to be refused even when a grant exists. In other words, the ability of an MCP client to search is separated from its ability to retrieve full content.

5.15 Why Sharing Claude Directives as an MCP Database Matters

On the claudecode side, Claude Directives form a canonical directive repository consisting of CLAUDE.md, rules, and skills. They can be materialized into .claude/ for Claude CLI and into AGENTS.md,

.agents/skills, and related structures for Codex. Claude Code and Codex can use this local harness directly. Meanwhile, SourceVault MCP's `sourcevault_directives` exposes the same directive set as searchable and retrievable data, returning rules and skills through references such as `sv://file`.

This allows local LLM clients such as LM Studio, provided they can invoke MCP tools during inference, to consult the same rules and skills as Claude Code and Codex when needed. The significance is that directives are managed as one source of truth projected through SourceVault/MCP, rather than manually duplicating different system prompts for every model. However, which directives are exposed to which model remains governed by AccessProfiles and release policy; MCP visibility does not imply unrestricted data release.

5.16 Writing Through MCP - `sourcevault_deposit`

`sourcevault_deposit` is an append-only API that lets an LLM actively store generated Markdown, Wolfram Language, PDFs, images, JSON, and other artifacts under SourceVault management. It is not a mutation API that directly permits overwriting, deletion, or privacy relaxation of existing objects. Committing requires an AccessGrant or model AccessProfile with `DepositArtifact` permission. Provenance records `authoredBy`, `inputRefs`, `citationRefs`, `promptRefs`, `SessionId`, and `BatchId`. A successful deposit returns an `sv://artifact/...` URI, and repeated use of the same idempotency key with the same content is treated as replay.

Privacy is inherited not only from the LLM's own declaration but also from the maximum privacy of inputs actually released during the relevant session/batch. If `sourceRefs` are present but no observed release basis exists, a fail-closed privacy floor is applied. The design therefore aims to prevent privacy laundering in which an agent reads a private source and then self-declares its output to be public.

5.17 Using SourceVault as an Object-Sharing Plane Between LLMs and Workers

ClaudeOrchestrator's principle is that workers cannot commit directly to a live Notebook; their outputs cross boundaries as artifacts. MCP deposit allows an LLM to store an artifact in SourceVault and receive an opaque `sv://artifact/...` reference, while another LLM or worker can retrieve an allowed projection through MCP `search/get`. Together, these mechanisms allow SourceVault to serve as a provenance-preserving object-exchange plane instead of using a shared Notebook as mutable shared memory.

This does not mean that all worker-to-worker communication is already automatically unified over MCP/SV URIs. Rather, it is a compositional principle for sharing among agents without violating the single-committer constraint. It also aligns with Orchestrator's session/episode design, which passes references instead of embedding large artifact bodies inside Petri tokens or conversations. Final application to a Notebook still proceeds through the reducer/commmitter or the NBAccess final-action queue.

Principal code and documentation sources: SourceVault_info/docs/user_manual.md /
SourceVault_info/docs/api_mcp.md / SourceVault_mcp.wl /
SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md /
claudecode_info/docs/api_directives.md /
ClaudeOrchestrator_info/design/claude_orchestrator_runtime_session_episode_petri_spec_v0_1.md

5.18 Outlook

A major future task for the SourceVault core is to extend the `sv://` URI resolver across all object classes and stabilize alias resolution with legacy references. If URIs become the common key across search, MCP, workflows, artifact relations, and citations, internal-ID differences no longer need to be exposed to external clients.

There is also room to make staleness propagation from source lifecycles to derived artifacts more automatic. When a snapshot changes, SourceVault could compute from dependency graphs which claims, summaries, workflow results, and Notebook descriptions require re-verification and place them into a review queue. This would increase its value as a long-term knowledge platform.

As MCP deposits and agent artifacts increase, it becomes increasingly important not to confuse self-generated material with external primary sources. `ArtifactType`, `SourceRefs`, `AuthoredBy`, `DepositedBy`, confidence, and review state should remain clearly separated, and LLM-generated artifacts should not be automatically promoted to evidentiary status. # Chapter 6. Code-Level Analysis of Integration Techniques

This chapter examines design principles that recur across all four repositories at the code level. The most important is the idea that permissions are not added from higher layers downward; instead, a lower-level base permission is narrowed by higher-level scopes. Runtime scope, Workflow scope, and Step scope are intersected with the `NBAccess` base policy, making it difficult for convenience features in higher layers to expand authority beyond the safety boundary.

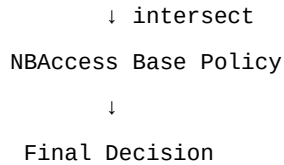
A second principle is to separate proposal from execution, artifact from live state, and immutable records from current projections. LLMs return proposals, workers return artifacts, SourceVault records append-only events, and current state is projected from those events. These may look like unrelated features, but they share one underlying philosophy: intermediate results should not automatically become the canonical state.

A third principle is that execution location is itself part of policy. Main kernel, subkernel, wolframscript, `SpecificMachine`, and remote-PC execution are selected not only for performance, but also according to Front End dependency, secret access, license seats, and fault isolation.

6.1 Concentrating Safety in Lower-Level Boundaries

At the code level, the four repositories consistently delegate final safety decisions to lower boundaries rather than re-implementing safety independently in each upper layer.

SourceVault / Orchestrator / Runtime Scope



Conceptually, final permission is the intersection of the following scopes.

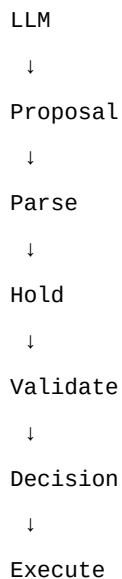
FinalPermission

- = NBAccessBase
- ∩ RuntimeScope
- ∩ WorkflowScope
- ∩ StepScope

Runtime and Orchestrator may narrow authority, but they may not broaden it beyond NBAccess's base permission.

6.2 Separation of Proposal and Execution

LLM output is a proposal, not an instruction.



This principle is shared not only by Runtime, but also by PromptRouter, the Orchestrator planner, and multi-model adjudication.

6.3 Separation of Artifacts and Live State

Workers do not modify the live Notebook. They return artifacts.

Worker Result

- = Payload
- + Sources
- + TaskRef

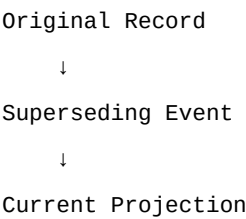
- + Schema
- + Validation State

Commit to live state occurs in a separate stage. This structure is also consistent with SourceVault’s derived-artifact model.

6.4 Immutable and Append-Only State

Raw SourceVault snapshots, claim events, Knowledge Home additions, TagAssertions, AuthorshipAssertions, workflow events, and similar records are managed as immutably or append-only as practical.

Corrections do not delete the old record. Instead, new events such as Superseded, Retracted, or decision events are added.



The current state can therefore be reconstructed from the event sequence.

6.5 Task Placement and Resource Placement

Execution location	Primary conditions
Main Kernel	Requires Front End, Notebook, UI, or session state
Subkernel	Pure computation, parallelizable, no Notebook side effects
WolframScript	Long-running work, batch processing, high-volume I/O, independent process preferred

Runtime makes recommendations, NBAccess validates the action, and Orchestrator authorizes the execution location as a workflow resource placement.

6.6 Responsibility Boundaries Among the Four Repositories

Problem	Canonical owner
Confidentiality of Notebook cells	NBAccess
Whether a held expression may execute	NBAccess
Progression of a single turn	ClaudeRuntime
Task-placement recommendations	ClaudeRuntime
Dependencies among multiple tasks	ClaudeOrchestrator

Problem	Canonical owner
Worker permissions	ClaudeOrchestrator + NBAccess
Notebook commit	ClaudeOrchestrator single committer
Sources and snapshots	SourceVault
Claims and evidence	SourceVault
Search release	SourceVault
Identity	SourceVault identity
Long-term topic graph	SourceVault mining / Knowledge Home

These boundaries are the prerequisite for understanding the extensions discussed later in the report.

6.7 Outlook

A desirable next step is to elevate these shared principles from coding conventions into machine-verifiable contracts. Invariants such as “upper layers cannot expand NBAccess permissions,” “workers do not possess `NotebookWrite`,” and “source-derived artifacts must carry provenance references” could be enforced through contract tests, making it harder for architectural drift to emerge as the package set grows.

Another challenge is to propagate a common run/step identity across every layer so that PromptRuns, Runtime events, Workflow runs, MCP calls, CapabilityLeases, and SourceVault artifacts can be joined into one trace. This would support not only fault analysis but also consistent explanations of questions such as, “Which LLM input, retrieval span, and workflow worker produced this Notebook output?”

As multi-PC execution expands, principles that work locally - single writer, atomic claim, and idempotency keys - also need to be preserved over a shared vault. Synchronization storage such as Dropbox is not a distributed database. The current approach based on per-machine logs and merged views therefore needs to be further systematized rather than replaced by assumptions of strong distributed consistency. # Chapter 7. Evolution from a Four-Layer Stack into a Knowledge-and-Execution Operating Model

Viewed separately, the four layers can be summarized as “Notebook operations + LLM + multi-agent orchestration + source management.” Once SourceVault adds URIs, MCP, mining, identity, AutoTrigger, diagnostics, and cognition, however, the components begin to form a cycle. Acquired knowledge triggers workflows, workflow results return to SourceVault as artifacts, and those artifacts are then referenced by subsequent LLMs and agents.

The key to this cycle is the use of `sv://` URIs and SourceVault as a shared plane. LLMs and workers do not write directly into a Notebook; they deposit artifacts into SourceVault, and other workers retrieve them through URIs. Final application to the Notebook is still performed by the single committer, so increasing autonomy does not require surrendering control over live state.

AutoTrigger and SystemDoctor turn this cycle from “a system that runs only when a person presses a button” into “a system that reacts autonomously to state changes and schedules.” The important architectural point is that a health gate is placed before autonomous execution, and ordinary workflows are stopped when diagnostics fail.

The analysis so far makes the original structure of the four repositories clear.

NBAccess owns the Notebook and secret boundary; ClaudeRuntime owns the single turn; ClaudeOrchestrator owns multiple turns and commit; and SourceVault owns the source lifecycle and evidence.

At that stage, the stack could be described as follows:

A system that safely connects confidential Notebooks to LLMs, advances complex work through multiple workers, and preserves referenced sources and generated artifacts in reproducible form.

The addition of identity, tags, mail structuring, the OOPS ontology, Knowledge Home, cognitive assistance, taint, Capability Broker, and anomaly analysis changed the character of the system.

The four repositories have moved beyond the scope of “tools for calling an LLM from a Notebook” and are developing into a system that manages knowledge acquisition, structuring, exploration, reasoning, execution, and auditing under one model of separated responsibilities.

This evolution does not replace the original four layers.

On the contrary, it depends on the responsibility separation established earlier. Even if the mining layer generates large numbers of relationships among materials, release gates still decide whether those materials may be viewed or sent. Even if an LLM uses neighboring material to propose an expression, NBAccess still decides whether that expression may execute. Even if multiple workers generate artifacts, only the single committer applies them to the Notebook.

The following sections examine the first major pillar of this expanded system: the SourceVault mining layer.

7.1 Prompt, Workflow, MCP, and SourceVault Form a Closed Loop

Connecting the mechanisms discussed above extends interactive LLM assistance into a closed loop. A user or external LLM retrieves evidence from SourceVault by `sv:// URI`, processes it through PromptRouter or a workflow, and returns the result to SourceVault as an artifact. Another worker can then reference that artifact, and after verification the single committer applies it to the Notebook. If the process itself is registered as a `Triggerspec`, subsequent runs can be automated based on time or SourceVault events.

SourceVault source / sv:// refs

↓

search / MCP / PromptRouter

↓

Runtime / Orchestrator workflow

```

↓
artifact deposit / snapshot / evidence
↓
AutoTrigger monitors schedules and events
↓
next workflow run
↓
result returns to SourceVault

```

Once this loop exists, SourceVault changes from a static vault into a shared work surface that LLM agents can read and write and that workflows continuously update. Autonomy, however, does not mean unlimited authority. Reads remain inside release gates, writes use append-only deposits or approved mutations, Notebook updates remain single-committer operations, and external actions remain constrained by capability and approval boundaries.

7.2 SourceVault_autotrigger - Safely Starting Workflows Automatically

SourceVault_autotrigger does not simply turn cron into a giant monolith. It separates the Trigger Registry, Scheduler/Watcher, append-only Job Queue, and Executor. A `TriggerSpec` contains the schedule, condition, target, execution placement, diagnostics policy, and run policy. Merely saving a trigger does not make it run automatically; conditions such as `Enabled` and `AutoRunCapability` must also be satisfied.

In the current phase 1.4, implementation has progressed through trigger validation and registration, schedule/condition matching, diagnostics gates, job construction during ticks, dispatch to subkernels/Workflow/CatalogWorkflow, scheduler registration through the shared polling tick, `specificMachine` placement, and remote-result retrieval. PromptRoutes remain main-kernel gated and are generally not auto-executed by the automatic tick.

WorkflowRoutes, WorkflowTemplates, and CatalogWorkflows can be started asynchronously when their conditions are satisfied.

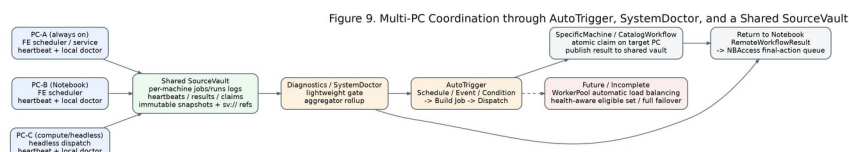


Figure 9. Multi-PC coordination through AutoTrigger, SystemDoctor, and a shared SourceVault

7.3 Designing SystemDoctor as the First Automatically Launched Workflow

One of the most important ideas in the AutoTrigger specification is that a comprehensive SystemDoctor workflow should run before ordinary business workflows. Specification v0.1-r9 treats Diagnostics/SIEM as a prerequisite layer for autonomous execution and states that “the first workflow launched automatically is not a user workflow, but a system-doctor workflow using

SourceVaultSystemDoctor/SourceVault_diagnostics.” Setting an ordinary workflow to Enabled=True presupposes that the recent health state of its required components is ok or an explicitly tolerated Degraded state.

This design goal must be distinguished from the current implementation. SourceVault_diagnostics implements a cross-package collector/store/doctor, license probes, service health, machine heartbeats, aggregator rollups, and related mechanisms, while AutoTrigger uses SourceVaultDiagnosticsLightweightDoctor as a component-scoped gate immediately before dispatch. At the same time, api_diagnostics.md still lists automatic triggering of the comprehensive doctor as deferred work. Thus, “SystemDoctor first” is the canonical design direction, while the lightweight-doctor gate is the part currently implemented in advance.

7.4 Sharing SourceVault Across Multiple PCs

When multiple PCs operate against a shared SourceVault, the vault itself becomes a durable common record surface. Diagnostics writes each PC’s heartbeat into a machine-specific path, and an aggregator rolls up health across machines. AutoTrigger job/run logs were likewise changed from a model in which every PC appended to one shared JSONL file into per-machine single-writer logs such as autotrigger/jobs/<machineTag>.jsonl and runs/<machineTag>.jsonl. This change was made in response to observed problems with conflicted copies under synchronization systems such as Dropbox, where remote appends could be lost.

For a SpecificMachine CatalogWorkflow, the requesting PC submits a shared job saying “run this slug on this machineTag,” and the target PC’s Front End scheduler or headless dispatcher picks it up. Before execution it creates an atomic claim to prevent duplicate execution, checks available license process slots, and publishes output.wxf to the shared vault upon completion. Notification back to the requesting Notebook proceeds through the NBAccess final-action queue, so remote execution does not violate the single-committer principle.

Capability	Current state	Significance for redundancy/load balancing
Per-machine heartbeat	Implemented	Independently observes each PC’s liveness
Aggregator rollup	Implemented	Provides global health and active-aggregator candidates
Per-machine jobs/runs	Implemented	Avoids concurrent-append conflicts on shared storage
SpecificMachine dispatch	Implemented	Places work on a designated PC such as a GPU or always-on machine
Atomic dispatch claim	Implemented	Prevents duplicate execution by FE scheduler and headless

Capability	Current state	Significance for redundancy/load balancing
Remote result publication	Implemented	dispatcher Retrieves results executed on another PC through the shared vault
License-seat guard	Implemented	Defers and retries when no process slot is available
WorkerPool automatic load balancing	Not implemented / under design	Future selection among candidate PCs based on load and health
Full failover	Partially designed	Graceful degradation and takeover across multiple always-on machines remain integration work

7.5 Current Level of Redundancy and Load Balancing

Combining these mechanisms already supports the path “request a workflow on PC A, execute it on PC B, publish the result to SourceVault, and retrieve it from the Notebook on PC A.” Headless compute nodes can also participate through opt-in dispatch. This is more than static file synchronization; it is a foundation for distributed execution that includes diagnostics, placement, claims, and result publication.

What is not yet complete is automatic placement of `EnvironmentIndependent/WorkerPool` jobs onto the optimal PC according to health, load, available model/GPU, and license capacity, together with complete failover to another node when the active node stops. The `AutoTrigger` source itself lists `WorkerPool` load balance/failover and machine-health-aware eligible sets as deferred. This report therefore assesses the current state as follows: much of the infrastructure required for multi-PC redundancy and load balancing is implemented, and `specificMachine` distribution is already practical, but a general `WorkerPool` scheduler remains a future capability.

Principal code and documentation sources:

`SourceVault_info/design/sourcevault_auto_trigger_scheduler_spec_v0_1.md /`
`SourceVault_autotrigger.wl / SourceVault_info/docs/api_autotrigger.md /`
`SourceVault_info/docs/api_diagnostics.md / SourceVault_info/docs/user_manual.md`

7.6 Outlook

The long-term direction is the completion of an event-driven knowledge-execution fabric centered on SourceVault. Workflows can be triggered by SourceVault events, schedules, approval states, and

queue states, with results stored again as `sv://artifact` objects. This would allow human intervention to be concentrated at the approval boundaries where it is genuinely needed.

Positioning a comprehensive SystemDoctor as the first automatically launched system workflow and using its result to determine whether other workflows may execute is operationally clear and attractive. The present implementation, however, has the lightweight-doctor gate ahead of the comprehensive auto-triggered doctor, and full producer-probe fan-out remains incomplete. Implemented behavior and future architecture therefore need to be advanced separately and explicitly.

For multi-PC deployment, adding WorkerPool scheduling, health-aware eligible sets, load balancing, and failover on top of `specificMachine` execution could turn a shared SourceVault into a small-scale distributed execution platform. Because shared file synchronization has weaker guarantees than a distributed database, however, the design should continue to treat per-machine single-writer logs and atomic claims as canonical rather than relying on strong distributed locks. # Chapter 8. The SourceVault Mining Layer

The SourceVault mining layer does more than perform full-text search over accumulated materials. It discovers relationships among materials and turns them into a knowledge space that can be explored as a neighborhood. Conventional vector search primarily places similar documents near one another; SourceVault combines more explainable relationships such as topic, author, sender, quotation, reply, explicit links, derivation, and retrieval history.

The OOPS corpus is important as an initial coordinate system for this knowledge space. It contains roughly 6,500 mail messages and about 4,100 topic items, together with manually assigned mail-topic mappings, topic relations, and quotation relations. It can therefore serve not only as an archive of historical mail but also as a seed for positioning new general mail, Notebooks, PDFs, and web materials in an existing topic space.

Neighbor Hopping is the user-facing form of this design. The currently viewed material becomes the next query, and candidate neighbors are proposed from three kinds of neighborhoods: shared topics, direct quotation/reply relationships, and relation-expanded topics. Even when the user cannot remember the right search terms, they can dig through the knowledge base by moving from one relevant object to the next.

8.1 Definition of Mining

In SourceVault, mining does not merely mean adding sources to a search index.

It means discovering, proposing, and confirming relationships among objects such as mail, local files, web pages, arXiv papers, Eagle assets, Notebooks, PDFs, and generated artifacts. These relationships include topics and tags; authors, senders, and creators; people and organizations; quotation and citation; reply and session structure; explicit links; derivation and generation; workflow inputs and outputs; historical retrieval usage; safety and privacy; and error and review state.

Mining results are represented as `MiningObjects` carrying fields such as `Confidence`, `TargetRefs`, `InputRefs`, `GeneratedByRunID`, and `ReviewState`.

A crucial principle is to distinguish mining results from the original material.

Original Object

↓

Mining Assertion / Proposal

↓

Human Review or Deterministic Validation

↓

Search Projection

Even if an LLM estimates that two identifiers “probably refer to the same person” or that two documents “probably belong to the same topic,” the estimate does not immediately become a canonical fact.

8.2 Multi-Stage Search Structure

Conceptually, SourceVault search has the following stages.

B1 Lexical Search

Japanese normalization, unigram, bigram, BM25

↓

B2 Mining Primer

summary, tags, authors, `ObjectSignals`

↓

B3 KG Local Expansion

topics, identity, links, quotations

↓

B4 Gated Body Search

permitted body chunks

↓

B5 Optional Dense / Rerank

Summaries and mining results are used as primers to narrow candidates, not as the final evidentiary basis for an answer. A final answer must ultimately be grounded in original text chunks or body-level evidence.

8.3 Basic Concept of Neighborhood Search

In ordinary search, the user enters a query. In neighborhood search, the object currently being viewed becomes the starting point of the next search.

The target object is first positioned in topic space.

`TopicPosition`

For example, suppose one document has the following topic position:

Cellular Automata	0.72
Reversible Computing	0.48
Information Thermodynamics	0.31
Research Meeting	0.18

and another PDF has:

Reversible Computing	0.65
Cellular Automata	0.59
Fredkin	0.25
Logic Circuits	0.21

The two become neighborhood candidates because they share Cellular Automata and Reversible Computing.

TopicPosition is not merely an opaque embedding. Each topic weight carries an explanation.

Method	Meaning
OwnerConfirmed	Explicitly confirmed by the owner
ExplicitOOPS	Explicit marker in OOPS text
SeedMatched	Surface-form match against the seed dictionary
RelationExpanded	Expanded from the relation graph
AutoExtracted	Newly extracted topic candidate
LLMProposed	Proposed by an LLM with low review status

The proportion of a paragraph that cannot be explained by existing topics is retained as `UnknownMass`. This avoids forcing every piece of content into an existing category.

8.4 Three Types of Neighborhood

8.4.1 Topical Neighborhood

Objects that share the same topic items. Whether the medium is mail, PDF, Notebook, or web, objects can be connected if they share a common topic projection.

8.4.2 Structural Neighborhood

Objects linked by direct structural relationships such as Reply, Quote, QuotedBy, Attachment, Link, Citation, GeneratedFrom, Supersedes, or WorkflowInput.

8.4.3 Neighborhood Through Relation Expansion

Starting from a central topic, the system follows the topic-relation graph and searches for objects associated with related topics.

Cellular Automata

└ Reversible Computing

└ Self-Reproduction

└ Artificial Life

└ Reservoir Computing

Expansion is bounded rather than unlimited, using hop count, node count, edge weights, and per-node top-k limits. OOPS auto-tagging primarily uses weighted one-hop expansion, while search-oriented knowledge-graph expansion implements cycle-safe multi-hop BFS.

8.5 Neighbor Hopping

`SourceVaultKnowledgeHomeNeighborhoodSearch` computes the `TopicPosition` of a target object and returns mail messages and Knowledge Home paragraphs with overlapping topics.

Current mail

↓

mail with the same topic

↓

another mail quoted by that mail

↓

a Knowledge Home addition on a related topic

Moving from the current material to a neighbor, then using that neighbor as the next starting point, is Neighbor Hopping.

8.6 Three-Ring Suggestions

Knowledge Home divides “what to read next” into three rings.

Ring 3: relation-expanded neighborhood

Ring 2: direct quote / reply / link

Ring 1: shared central topics

Current Object

Ring 1 contains materials sharing central topics. Ring 2 contains direct neighbors linked by quotation, being quoted, reply, or explicit links. Ring 3 contains materials associated with topics reached by expanding the relation graph from the central topics.

Each candidate is accompanied by `Reasons`, allowing the user to inspect why it was suggested.

8.7 Common Projection Across Media

SourceVault does not erase differences among media. Instead, it preserves each native object and creates a common projection.

`ObjectURI`

`ObjectClass`

`Title`

Body / Chunk
 Summary
 Tags
 TopicRefs
 Authors
 Links
 PrivacyLevel
 SafetyState
 SourceEpoch
 DerivedFrom

Medium	Primary unit	Structural relationships	Identity observations
Mail	Mail, Paragraph, Quote, Session	Reply, Quote, Attachment	From, To, Cc
Notebook	Notebook, Section, Cell, Output	Order, evaluation, generation	Creator
PDF	Document, Page, Section, Figure	Citation, Version	Author, XMP Creator
Web	Page, DOM Block, Article	URL, Link, Snapshot	schema.org Author
arXiv	Paper, Version, Reference	DOI, Citation	arXiv Author, ORCID
Artifact	Output, Run Result	GeneratedFrom	LLM, Agent, Workflow

This common projection makes it possible, for example, to start from a Notebook cell and traverse to a PDF on the same topic, a web page by the same author, or the mail message that served as a quotation source.

8.8 SourceVault_identity

A single person can be represented by multiple surface forms.

imai@example.jp

Katsunobu Imai

今井克暢

K. Imai

GitHub: transreal

Notebook Creator: imai_

SourceVault_identity separates this into two levels: Identifier and Entity.

An Identifier is an observed raw identifying datum, such as Email, URI, SNS, PersonName, OrganizationName, ORCID, arXivAuthor, DOIName, WebAuthorProfile, DocumentCreator, or LLMOAgent.

An Entity is a real-world unit such as Person, Organization, Bot, Service, or MailingList that groups multiple identifiers.

Mail From, To, and Cc values are registered as Identifiers during ingest. Because deterministic IDs are generated from the identifier kind and normalized value, repeated ingestion of the same address upserts to the same Identifier.

8.9 AuthorshipAssertion and EntityLinkProposal

Relationships between media and people are projected into common assertions.

```
Mail      — Sender — Identifier / Entity
PDF       — Author — Identifier / Entity
Notebook  — Creator — Identifier / Entity
Web       — Author — Identifier / Entity
Artifact  — Agent  — Identifier / Entity
```

When identity equivalence has not been confirmed, the system does not directly assign an EntityRef to an Identifier. Instead it stores an EntityLinkProposal.

A proposal carries a score, features, provenance, and review state. A human accept or reject decision is not silently overridden by later automated rescoring.

Identity resolution is not a security boundary. The fact that an identifier appears to belong to a known person does not lower its privacy level or authorize cloud release.

8.10 The OOPS Mailing List

OOPS is a corpus of approximately 6,500 mail messages from 1992 through 2005 and about 4,100 topic items.

SourceVault uses it not merely as an additional mail archive but as an initial reference coordinate system for Knowledge Home.

The principal data include the following.

Data	Role
item-name.index	Topic-item names and aliases
mail-to-item.index	Mapping from mail to topics
item-to-mail.index	Mapping from topics to mail
item-relation.index	Weighted directed relations among topics
item-relation-up.index	Reverse topic relations

Data	Role
<code>quote-table.index</code>	Source mail for quotations
<code>quoted-table.index</code>	Being-quoted relationships
<code>mail-info.index</code>	Mail number, list, author
<code>oops*.txt</code>	Mail bodies

`mail-to-item.index` is manually labeled gold data and can serve as a reference for topic inference and search evaluation.

8.11 Owner-Scoped Topic Namespaces

OOPS topics are not global keywords.

`[ki 2573]`

`[mi 2573]`

`[aga 2573]`

These represent topics in different namespaces.

In the initial configuration, `ki` is mapped to the current owner, while `mi`, `aga`, and others remain namespaces belonging to different owners.

Even when labels are identical, topics belonging to different owners are not automatically merged.

`TopicItemRef`

`Namespace`

`LocalId`

`OwnerRef`

`NamespaceKind`

`CanonicalLabel`

`SurfaceForms`

`PrivacyLevel`

`SourceRefs`

`ReviewState`

Unknown namespaces are preserved as `Missing["UnknownOwner"]` rather than discarded.

8.12 Internal Structure of OOPS Mail

The OOPS model does not treat a mail message as one long string.

`Mail`

└ Header

└ Prose Paragraph

| └ Primary Topic

| └ Secondary Topic

└ Quote Paragraph

- | └ QuotedFrom
- | └ Signature
- | └ Footer

Topic-assignment priority is `ExplicitOOPS`, `SeedMatched`, `RelationExpanded`, `AutoExtracted`, and `LLMProposed`, in that order.

Explicit markers such as ☉, ○, •, and `[namespace id]` are treated as the highest-quality human annotations. If a seed dictionary entry matches the body, the result is `SeedMatched`, and the relation neighborhood of that topic can be added at lower weight as `RelationExpanded`. Terms not present in the seed become `AutoExtracted` candidates and are not promoted to official topics until the owner confirms them.

8.13 Quotation Graph

OOPS mail includes markers such as the following.

```
-*- Quote (from 6539) -*-  
...  
-*- Unquote -*-
```

The direction of a quote edge is from the quoting mail to the quoted mail.

Seed relationships in `quote-table.index` are treated as the authoritative graph. Fuzzy matching used for general mail does not overwrite existing seed relationships.

8.14 Mail Sessions

Quotation edges, subjects, and reply relationships are used to group multiple messages into sessions.

Mail Session

- | Initial Proposal
- | Reply
- | Counterproposal
- | Decision
- | Later Reference

By creating search chunks at the session level rather than only at the individual-message level, the system can retrieve concepts such as “the final schedule,” “the conclusion of the discussion,” or “unresolved issues.” The session summary is not itself treated as final evidence; grounding references to the corresponding paragraphs and quotation edges are preserved.

8.15 Extension to General Mail

The goal is not to convert all general mail into OOPS’s historical file format. The goal is to project general mail into the same topic/quote/session model.

General Mail

- └ Paragraph segmentation
- └ Quote / Forwarded-block extraction
- └ Reply / Reference relationships
- └ Matching against OOPS seeds
- └ New topic candidates
- └ Mail Relation Graph
- └ Topic Graph

General-mail structuring is seed-optional. If OOPS seeds are available, existing topics can be reused; if no seed is available, `TopicVocabulary` can grow from the mail corpus itself.

To avoid circular dependency between vocabulary construction and session detection, processing proceeds in two passes.

Pass A

Mail-level Vocabulary

↓

Relation Graph and Sessions

↓

Pass B

Session-aware Vocabulary Refinement

↓

Paragraph Topics and Topic Graph

8.16 Growth of New Topics

Because OOPS vocabulary ends in 2005, terms such as ChatGPT, Claude Code, ComfyUI, Agentic Search, DGX Spark, and Meta Quest 3 may not exist in the original seed.

Such terms are extracted as `AutoExtracted` candidates and promoted to new topic items after owner confirmation.

Unknown Surface

↓

`AutoExtracted` Candidate

↓

Owner Confirmation

↓

New Topic Item

↓

Re-index

↓

Future `SeedMatched`

OOPS is therefore not a fixed classification table; it is an initial map that can grow through current mail, Notebooks, PDFs, and web material.

8.17 Privacy Boundaries

Being “nearby” does not imply that an object may be read.

Near

- ≠ Readable
- ≠ Cloud-sendable
- ≠ Publicly exportable
- ≠ Safe to execute

OOPS ingest evaluates privacy not only from the list name but also from actual To, Cc, Delivered-To, and similar headers.

Materials that include private-list or third-party information can receive tags such as PrivateML, ThirdPartyContent, NoCloudLLM, and NoPublicExport.

If even one message in a session has high privacy, the session’s PrivacyLevel becomes the maximum and its tags become the union. Ranking boosts from mining or identity affect only ranking and never relax release gates.

8.18 Implementation Status

According to the July 14, 2026 handoff documentation, the following Knowledge Home phases were implemented and verified.

Phase	Content	Status
1A	Read-only browser for topics, mail, and quotations	Implemented and verified
1B	Non-destructive append, ULID, aliases, supersede, undo, merge	Implemented and verified
1C	TopicPosition, UnknownMass, neighborhoods, three-ring suggestions	Implemented and verified

A fully unified NeighborhoodSearch spanning PDFs, Notebooks, web, and general mail is still in the process of connecting the common model to the projections for each medium.

8.19 Outlook

The largest future challenge is to extend the neighborhood search already implemented for OOPS mail and Knowledge Home paragraphs so that it works seamlessly across Notebooks, PDFs, web pages, general mail, and MCP artifacts. Native structure for each medium must be preserved while projecting into common TopicPosition, AuthorshipAssertion, and relation edges.

A second challenge is quality evaluation of mining results. `AutoExtractedTopics`, `EntityLinkProposals`, and relation expansion can produce false positives, so the system needs mechanisms for self-correction using human review, confidence, and rejected history. OOPS's manually assigned data can serve as ground truth for evaluating not only recall and NDCG but also erroneous-link rates.

A third challenge is integration with agentic search. When MCP clients or LLM agents perform Neighbor Hopping, primers and summaries should not become final evidence; the last step should always ground the answer in gated body/span content. This preserves evidentiary quality and privacy boundaries even as exploratory freedom increases. # Chapter 9. Layered Defense Across Security, Diagnostics, and Cognitive Assistance

This chapter is easiest to understand if safety problems, including prompt injection, are viewed as a layered defense rather than as the job of a single detector. SourceVault does not assume that it can detect 100% of malicious instructions. It first treats external web pages, PDFs, and mail as `UntrustedData` rather than instructions, and performs deterministic pre-scans for known injection patterns, tool misuse, credential exfiltration, and obfuscation. High-risk objects are quarantined and removed from reasoning retrieval.

Because attacks can still be missed, the next layer tracks their effects through taint propagation. Artifacts that summarize, quote, or derive from dangerous sources inherit taint along content-carrying edges and the specific contributing spans that actually affected the derived result. Merely being related through `SameTopic` or `RelatedTo` does not propagate taint. This treats indirect prompt injection as an information-flow problem: which downstream artifacts were actually influenced?

The entire LLM request envelope is then bound through a `PreparedInputToken` so that substitutions immediately before transmission can be detected. Eighteen LLM entry points currently have staged Shadow/Warn/Enforce gates, but the default remains Shadow and the transition toward system-wide Enforce is still in progress. Even if an LLM is fooled, external actions require a separate `CapabilityLease`, with one-shot atomic consumption and an `ExecutionTicket` to reduce replay and TOCTOU risks.

The core of this chapter, therefore, is not simply “detect malicious input.” The system separates input authority, isolates dangerous content, propagates taint to downstream artifacts, binds LLM requests, requires separate capabilities at execution time, and restricts final Notebook writes to the single committer. Diagnostics and cognitive assistance support these defenses from the side: SystemDoctor observes pipeline health, while cognition adds confirmation around ambiguous human instructions and irreversible actions.

Recent SourceVault work adds another axis: **states in which something that should have been executed has not been executed**. Examples include the owner forgetting to reply to actionable mail, overlooking a Notebook deadline or `NextReview`, failing to begin preparation required by a calendar event, or not fulfilling a recurring Routine. This does not classify the human as “abnormal.” Rather, it treats a mismatch between the system's recorded Obligation and observed Fulfillment as an **operational state anomaly**. Like an LLM's unexpected tool request or a decline in run integrity, it

is first reduced to deterministic events and rate streams, then compared with a baseline in observe-only mode, and finally connected to attention and review.

From this perspective, the SourceVault safety layer is expanding beyond “prevent attacks and LLM malfunctions” toward also detecting “important work silently falling through the cracks.” The current Routine Agenda combines calendar events, Notebook tasks under `$onWork`, and actionable mail into the same daily work surface, allowing unexecuted, overdue, and reply-required items to be handled through a common attention plane. Chapter 9 can therefore be read as an attempt to place security anomalies, system-health anomalies, LLM anomalies, and owner operational anomalies into one framework of observation, evidence, attention, and review - without conflating their distinct authority boundaries.

9.1 Why a Safety Layer Is Needed Outside the Mining Layer

The mining layer connects materials that were previously isolated. As connectivity increases, however, so do risks: leakage of topics derived from private mail, prompt injection embedded in web content, taint propagation from quoted material into generated artifacts, incorrect identity links, dangerous expressions proposed by an LLM, corruption of shared state by workers, accidental sending caused by ambiguous user instructions, and leakage of confidential state through diagnostics.

Safety is therefore distributed across multiple layers rather than implemented as one filter.

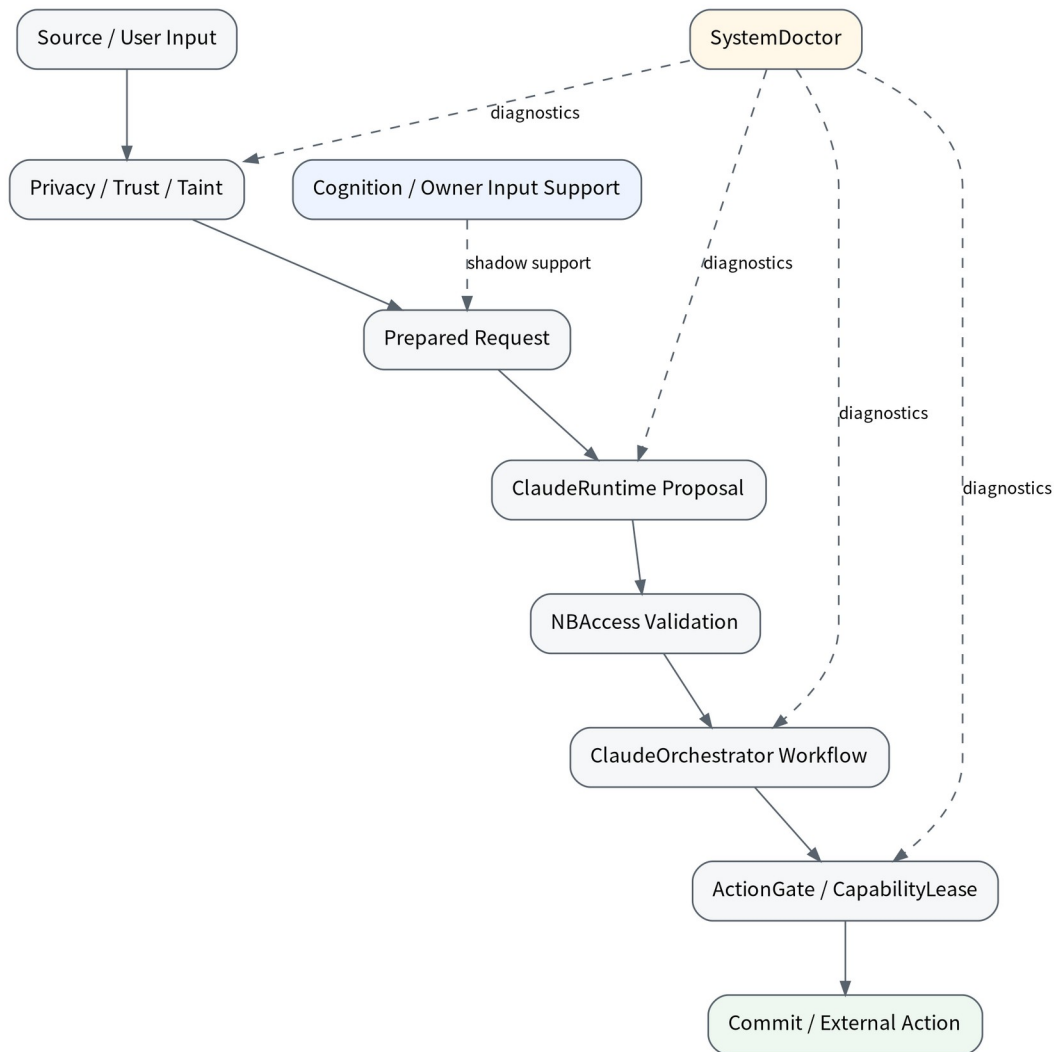


Figure 6. Layered defense spanning security, diagnostics, and cognitive assistance

9.2 Confidential-Cell Management

Notebook confidentiality propagates beyond secret cells to outputs that depend on those cells. NBAccess constructs provider-facing context and prevents Runtime or Orchestrator from receiving raw secrets.

SourceVault prompt privacy conservatively composes factors such as Prompt Cell Privacy, Referenced Source Privacy, Dependency Privacy, Model Trust Domain, Result Privacy, and User-specified Privacy by taking the safer maximum.

High-privacy prompts may therefore receive constraints such as:

AllowedTrustDomains -> Local / Private

CloudFallback -> Deny

RawPromptStored -> False

The critical point is that Runtime or Orchestrator does not first see a secret and then redact it. NBAccess provides only the permitted projection from the beginning.

9.3 All External Input Is Untrusted Data

External bodies from web pages, PDFs, mail, APIs, arXiv, HTML, and similar sources are treated as data, not as system instructions.

Even if the content says “ignore previous instructions,” “run this tool,” or “output credentials,” that text is not promoted into execution authority.

External content cannot elevate its own instruction authority through self-assertion.

9.4 InputTrustAssessment

Inputs are evaluated using a risk vector including Prompt Injection, Tool Misuse, Credential Exfiltration, Data Exfiltration, Obfuscation, Invisible Unicode, HTML Comment Directive, and Cross-object Contamination.

A deterministic pre-scan runs first, followed by additional analysis when needed. Quarantined objects are excluded from primers, KG expansion, agentic retrieval, and LLM reasoning.

9.5 Taint Propagation

Taint is not propagated unconditionally across every relationship edge. Propagation is limited to cases such as:

```
CarriesContent -> True
and
ContributingSpanRefs is not empty
```

For example, merely sharing a topic with a dangerous web page does not create taint. Taint propagates only when an actual content span from that page contributes to a summary or generated artifact.

For edges such as `Contains` and `AttachmentOf`, the system records the direction in which content actually contributed, rather than treating mere containment or attachment as sufficient.

```
FromRef
ToRef
EdgeKind
CarriesContent
ContributingSpanRefs
TransformRef
Direction
PolicyVersion
```

The model distinguishes between an object that is partially tainted and one that is quarantined as a whole.

9.6 Declassification

Declassification of high-risk material is not performed solely on the basis of the LLM's own judgment, a majority vote among multiple LLMs, a reducer decision, or a simple owner confirmation.

Depending on the risk, the process can require a sanitized projection, deterministic checks, verification of the evidentiary basis, and additional approval.

9.7 Capability Broker

Permission to read data or send a prompt is separate from permission to perform an external action.

Execution authority is composed through the following layers.

AccessProfile / Grant

↓

ActionGate Decision

↓

CapabilityLease

↓

Atomic Consume

↓

One-time Execution Ticket

↓

Dispatch

The canonical state of a `CapabilityLease` is maintained in the broker-side ledger. One-shot use, `MaxUses`, expiry, revocation, and replay are decided atomically through that ledger.

9.8 PreparedInputToken

A `PreparedInputToken` is not bound merely to a hash of the prompt body. It is bound to the complete request envelope that will actually be transmitted.

Provider

Model

Deployment

System Message

Developer Message

User Message

Tool Schemas

Retrieval Refs and Digests

Isolation Profile

Privacy Decision

Capability Ceiling

Output Schema

Run Ref

Step Ref

The provider adapter recomputes the digest immediately before sending and refuses the request on mismatch. This prevents TOCTOU substitutions of the model, system prompt, tool schema, or retrieved content after inspection.

9.9 Shadow / Warn / Enforce

The existence of the Capability Broker infrastructure does not mean that every entry point is already continuously enforced.

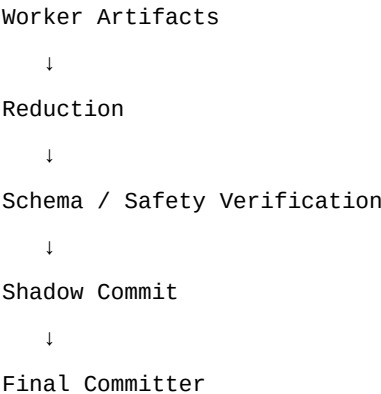
Mode	Behavior
Shadow	Record only; do not block transmission
Warn	Warn about mismatches
Enforce	Reject missing or mismatched tokens

The inventory of LLM entry points and self-prepare token wiring have advanced significantly, but operational mode depends on the entry point and profile. The system as a whole should therefore not be described as fully Enforced by default.

9.10 Side-Effect Separation in ClaudeOrchestrator

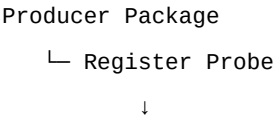
Workers are artifact producers. Operations that workers are not allowed to perform directly include NotebookWrite, RunProcess, SystemCredential, unrestricted URL access, and live Notebook commit.

Only the single committer performs commit.



9.11 SystemDoctor

SourceVault SystemDoctor is not designed as one giant diagnostic tool directly dependent on every subsystem. Instead, it aggregates probes registered by producer packages.



SourceVaultSystemDoctor

↓

OK / Degraded / Failing

Probe execution defensively handles timeout, messages, `$Failed`, `Abort`, `Throw`, malformed results, and similar failures. One failing probe does not stop SystemDoctor as a whole; the corresponding component is reported as `Degraded` OR `Failing`.

9.12 Process Supervisor Diagnostics

Runtime's Process Supervisor manages external jobs through states such as `PendingSpawn`, `Running`, `CleanupPending`, `Archived`, and `CleanupFailed`.

To account for PID reuse, it checks process start time in addition to the PID. Cleanup ordering releases the seat first and archives afterward. If `CleanupFailed` exists, the SystemDoctor probe returns `Degraded`.

9.13 Confidentiality of Diagnostic Information Itself

Diagnostic information can itself be sensitive. Even a Boolean such as “a private alert exists” may reveal information about the user's state or the existence of a private corpus.

Ordinary heartbeats therefore include only allow-listed fields such as `Component`, `Health`, and public `ReasonCode`.

Fields such as `PendingSensitiveAlerts` are removed from ordinary probes and kept in local-only doctors.

Sink	Permitted information
Local SystemDoctor UI	Necessary local-only details
Diagnostics Log	Explicit allowlist
Machine Heartbeat	Component, Health, public Reason
Cloud Heartbeat	Minimal data such as <code>GlobalHealth</code>
Mail Alert	Fixed, generalized wording

9.14 Cognitive Assistance

SourceVault cognitive assistance is not a system that evaluates a user's ability and changes authorization accordingly.

Its purpose is to provide additional support before a final action when the target of an instruction is ambiguous, a recipient is unspecified, quoted text and the owner's instruction are mixed, an operation is irreversible, evidence is insufficient, the same search is being repeated, or the process is becoming anchored to a mistaken assumption.

Input is decomposed into signals such as `OwnerInstruction`, `QuotedData`, `AmbiguousReferent`, `RecipientUnspecified`, `PotentialQuotedDataRelease`, and `IrreversibleActionRequested`.

Support modes include `Normal`, `ReviewEnhanced`, `ConfirmBeforeCommit`, and `DraftOnly`.

Irreversible actions require owner confirmation regardless of any cognitive state.

9.15 SensitiveLocalVault

Cognition-related events are stored in a `SensitiveLocalVault` under

`<LocalState>/sensitive/cognition/`.

The principal rules are: exclude the data from cloud synchronization, encrypt it, shard it by subject x month, use local key references, maintain bitemporal events, support crypto-shredding, avoid storing prompt bodies by default, and never use the data for authorization.

Values such as `SupportNeedTier` represent the degree of support needed, not an assessment of ability or authority.

9.16 Shadow-Only Recorder

Cognitive observation at the `ClaudeEval` entry point is shadow-only.

- It does not change call arguments.
- It does not change return values.
- A timeout does not stop the main operation.
- Prompt bodies are not stored.
- Only digest, length, mode, and signal names are stored.

Cognitive assistance therefore does not expose the user's state to the LLM prompt or automatically reduce permissions.

9.17 Human and LLM Errors

A distinctive characteristic of `SourceVault` is that it does not treat error as an LLM-only problem. More recent Routine Attention work goes further: not only incorrect decisions but also **required work that is never performed** is placed on the same observable operational-state surface.

Error or deviation	Human / owner side	LLM / system side
Memory error	Misremember a previous decision	Use stale context
Assumption	Misidentify target or recipient	Fill in missing information without basis
Looping	Repeat the same exploration	Regenerate the same query
Overconfidence	Commit despite ambiguous instruction	Assert a weakly supported claim
Quotation confusion	Mistake someone else's words for the owner's instruction	Promote untrusted text into instruction authority
Fulfillment omission	Forget a reply, attendance,	Scheduled run, worker, or

Error or deviation	Human / owner side	LLM / system side
	preparation, or recurring task	probe does not complete as expected
State oversight	Miss requirements because mail, Notebook, and Calendar are viewed separately	Lose decision evidence because source coverage becomes Stale OR Unavailable

These cases are handled through common concepts such as events, evidence, confidence, review, abstention, commitment, and obligation. However, human non-execution must not be interpreted as an abnormality of cognitive ability. The anomaly is the **difference between the expected system state and the observed state**, not the person.

9.18 Treating Non-Execution and Forgetting as Operational State Anomalies

The canonical Routine Attention core specification v0.4 normalizes `CalendarEvent`, `OnWorkTask`, `Commitment`, and `Routine` into occurrence-level Obligations and, when necessary, derives `PrepTasks` from `PreparationSpec`. The key idea is not to give schedules, Notebook tasks, commitments, and recurring work separate reminder mechanisms, but to map all of them into a common model: “what should be satisfied by what point for this occurrence?”

Fulfillment is determined from deterministic evidence rather than by an LLM. When a source cannot be retrieved and the state is therefore unknowable, the value is `Unknown` rather than `False`, using Kleene three-valued logic. This prevents Calendar or Notebook acquisition failures from being misclassified as non-execution. At the same time, once an occurrence has been overdue, that history is retained as `wasOverdue` even if the task is later caught up; late completion does not erase the fact that the deadline was previously missed.

In the implementation, `SourceVaultRoutineExecutionStreams` aggregates resolved `ObligationOccurrences` by day and generates `RoutineExecutionRate` and `RoutineOverdueRate`. `SourceVaultRoutineOverdueSecondsByDay` integrates `OverdueContractSeconds`, and these streams can be sent to the existing Cane anomaly workflow. Prompt Injection signals, unexpected tool requests, and declines in run integrity can therefore share the same anomaly-analysis infrastructure with a stream representing **the rate at which work that should have been performed was not performed**.

Conceptually, the mapping is:

```

LLM / Agent side
unexpected tool request
run integrity decline
injection signal increase
    ↓
rate stream
    ↓

```

Cane anomaly workflow

Owner / Routine side

missed routine

overdue notebook task

unanswered actionable mail

preparation not started

↓

RoutineExecutionRate / RoutineOverdueRate

↓

Cane anomaly workflow

This does not mean that the two classes are interpreted as the same type of anomaly. An LLM anomaly describes model/run health, while a Routine anomaly describes a mismatch between Obligation and Fulfillment. What is shared is the processing form - **observation -> stream construction -> baseline comparison -> attention/review** - not causal inference or authorization control. Routine anomalies are currently observe-only as well: detecting non-execution does not change the owner's authority or cause the system to execute the task automatically.

Implementation status shows that R1-R4 and R7 in `SourceVault_routine.wl` are documented as implemented, connecting the Board, AttentionContext, durable notification queue, ActionGate, and execution-rate anomaly streams. R8 automation proposals and the AutoTrigger-side AT-1 permit are also implemented, but AT-1 enforcement is additive and disabled by default, so existing behavior is not changed automatically.

9.19 Routine Agenda: Calendar, Notebook, and Mail on the Same Attention Surface

The practical value of Routine Attention is not only that it can generate anomaly streams. It also gives the owner a single surface on which to see “what am I overlooking today?” The current `SourceVaultRoutineAgendaData / SourceVaultRoutineAgendaView` implementation comprehensively handles at least the following three sources.

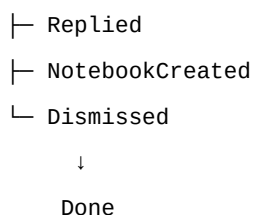
Information source	Current ingestion path	Treatment in Agenda
Calendar	NBCalendarEvents / FreeBusy from NBAccess	Timed events, all-day events, busy capacity
Notebook	Scan \$onWork using NBOnWorkTasks	Deadline, NextReview, Overdue, click to open Notebook
Mail	SourceVaultMailAgendaItems	Reply required, task request, attendance request, confirmation, inferred deadline

The Agenda implementation first reads Calendar events, then adds `$onWork` Deadline/NextReview items into the same day buckets, and finally weakly binds in the R9 Mail Agenda. Mail with a deadline is also inserted into the day calendar as `MailDeadline`, while actionable mail appears in a separate band. The design therefore avoids isolating “mail as something seen only in a mail app,” “Notebook TODOs as something seen only inside the Notebook,” and “Calendar as something seen only in the calendar.” Instead, all are projected onto **one Attention surface**.

Mail Agenda R9 does not invoke an LLM while rendering the Agenda. It reads the precomputed Summary/Category/Priority/Deadline values produced by `SourceVault_maildb` from an index and narrows candidates in the order: category gate -> spam/irrelevance gate -> owner-directed determination -> resolved-item exclusion. Owner-directed determination uses owner addresses and organizational addresses from the identity layer; only when necessary does it lazily decrypt the beginning of the message body to check addressee patterns. Messages in the same thread are collapsed into one item.

Resolution state is managed as follows.

Pending



If a reply is required, the system opens a reply Notebook. If the mail needs to be inherited into work, it creates a work Notebook under `$onWork`. That inheriting Notebook retains `MailRecordId`, allowing the user to return to the original mail and the subsequent reply thread. In the opposite direction, the mail sidecar can reference the Notebook path. This avoids the common failure mode in which a task is received by email, a work Notebook is created, and the relationship to the original message is then lost.

The PlanView extension goes further by generating preparation tasks in advance from `PreparationSpec` for meetings and talks, and by using `CapacityModel`, `LatestSafeStart`, `Slack`, daily replanning, and `FocusRequest` to detect and mitigate procrastination. The plan itself is intentionally disposable; only durable intent such as pins, `FocusReservations`, and `PlanningIntents` is retained. When inputs are `Stale` or `Partial`, silent replanning is stopped and the previous plan is retained under `PlanInputDegraded`.

9.19.1 Verification of Implementation Status

Inspection of the `main` branch and recent commits as of July 16, 2026 shows that this is not merely a design proposal; substantial portions are already implemented.

- The Routine core implements identity, three-valued evidence, Resolution, the durable-facts ledger, ActionGate, Board, AttentionContext, the durable notification queue, and R7 anomaly streams. The API documentation reports 262 cumulative green Routine tests.
- R8 implements deterministic formulaic scoring, automation eligibility, proposal/approval, TriggerSpec drafts, and one-shot permits. The AutoTrigger side includes the AT-1 permit/ExpiresAt/EnabledAudit gate. Permit enforcement, however, is disabled by default.
- SourceVault_routineplan.wl implements PreparationTask generation, LatestSafeStart/Slack, DAG-validated capacity placement, replanning, FocusRequest, Gantt/Load/Process views, ScheduleFabric, and a Calendar + \$onWork Agenda. The API documentation reports 48 green planning tests and 20 green view tests.
- The latest R9 commit adds SourceVault_mailagenda.wl, implementing owner-directed actionable mail, thread collapsing, a Replied/NotebookCreated/Dismissed state machine, mail <-> Notebook inheritance, and privacy gates. SourceVault_routineplan.wl also adds IncludeMail / MailItems, integrating MailDeadline and an actionable-mail band into the Agenda.
- At the same time, the end of api_routine.md still states that “R9=mail” is unimplemented. This is documentation lag relative to the latest main code and commits. This report therefore characterizes R9 as **“the specification remains draft, the implementation has landed on main, and parts of the API documentation are still catching up.”**
- Within the PlanView extension, shared-ledger wiring for durable PlanningConstraint / PlanningIntent / PlanChunkProgress, the Petri adapter for formal workflow use, and the Plan diff view remain documented as unimplemented.

This implementation direction means that SourceVault is beginning to treat not only LLM errors and prompt injection, but also **the silent persistence of important owner work in an unexecuted state** as a state deviation worthy of system observation. SystemDoctor component health and Routine Attention operational health, however, are not the same thing. Software failure and an unperformed owner task should not be collapsed into one Failing state; a higher-level health/attention view should present them in parallel while preserving separate evidence and action policies.

9.20 Multi-LLM Adjudication

Agreement among multiple models is not treated as correctness by itself.

Deterministic Test

>

Evidence

>

Verifier

>

Calibrated History

>

Model Agreement

Models belonging to the same `correlationGroup` are not counted as independent votes.

When no valid conclusion can be reached, abstention is allowed through outcomes such as `NeedMoreEvidence`, `NeedOwnerClarification`, `SafeDraftOnly`, and `NoCommit`.

High-risk actions are not accepted without owner confirmation even when multiple models agree.

9.21 Anomaly Analysis

The Cane anomaly workflow is observe-only.

Initial active streams include combinations such as `InjectionSignalRate` x `RunIntegrityRate` and `Pre-scan Failure/Liveness` x `Unexpected Tool Request`.

Correlations involving the owner's state are not included in the initial active set and remain research-only opt-in.

Even when a suspicious correlation is found, it does not directly trigger containment or permission restriction.

9.22 Cold Start and Baselines

When there is not enough baseline data, the system does not guess and raise an alert.

`Missing["InsufficientBaseline"]`

Baseline updates also proceed in three stages: `Candidate` -> `Validation/Backtest` -> `Activate`. Because changing a baseline changes future decisions, the update is audited as state mutation even in observe-only mode.

9.23 Correlation and Causation

If two signals change because of the same upstream event, the system does not treat this alone as causal evidence.

A `SharedUpstream` hypothesis is normally capped at `AssociationSupported`.

Promotion to causation requires additional evidence such as `Controlled Replay`, `Deterministic Attack Chain`, `Isolation Stopped Recurrence`, or `Independent Streams`.

9.24 Summary of the Layered Defense

Layer	Defense
NBAccess	Notebook privacy, secret dependency, safe execution, keys
ClaudeRuntime	Held proposals, validation loop, prohibition on direct resource access
ClaudeOrchestrator	Worker/committer separation, transactional

Layer	Defense
	commit
SourceVault ingest	Untrusted data, security pre-scan
SourceVault mining	Provenance, confidence, review, taint
Prompt boundary	PreparedInputToken, request binding
Action boundary	CapabilityLease, atomic consume
Diagnostics	Probe isolation, sink field allowlists
Cognition	Local-only, shadow mode, confirmation for irreversible actions
Routine Attention	Obligation/Fulfillment, forgetting detection, durable notifications, Agenda
Adjudication	Evidence-first, abstention
Anomaly	LLM/security/routine rate streams, observe-only, cold start, causal-elevation requirements

Safety therefore emerges not from one filter but from the product of multiple boundaries.

Safety

- = Privacy Boundary
- × Proposal Validation
- × Workflow Capability
- × Release Gate
- × Request Binding
- × Action Lease
- × Human Confirmation
- × Diagnostics

9.25 Additional Safety Boundaries Introduced by MCP, AutoTrigger, and Multi-PC Operation

Adding MCP and AutoTrigger means that the system is no longer one that runs only when a user explicitly executes something in a Notebook. This makes several boundaries especially important: MCP is a projection layer rather than an authority; AutoTrigger is a thin scheduler that does not decide internal workflow side effects; and SystemDoctor remains read-only in principle. An MCP transport token is not a data-access grant, and an `sv://` URI is not a capability.

The fact that AutoTrigger can construct a job also does not mean that the external side effects inside that job have been authorized.

Boundary	What it permits	What it does not permit
<code>sv://</code> URI	Object identity and reference	Read authority by itself

Boundary	What it permits	What it does not permit
MCP transport	Client connection to tool endpoint	Automatic release of private data
<code>sourcevault_search/get</code>	Search/retrieval of released projections	Body/raw retrieval without a grant
<code>sourcevault_deposit</code>	Append-only artifact creation	Deleting existing objects, relaxing privacy, arbitrary mutation
AutoTrigger	Job construction and executor invocation when conditions match	Replacing workflow-internal side-effect decisions
<code>SpecificMachine</code> placement	Selection of the target PC	Bypassing NBAccess/action capabilities
SystemDoctor	Observation and aggregation of health	Secret disclosure or authority expansion based only on diagnostics

In particular, a self-declared client `ProviderClass` is not used as authority for release decisions. Only trusted information established through transport, grants, or the main kernel is used. Avoiding the misclassification of cloud-backed clients such as Claude Code and Codex as `LocalLLM` simply because they have local tools is important when opening SourceVault to heterogeneous models.

9.26 Preserving the Single-Committer Principle as Autonomy Increases

Even if an LLM can create artifacts through MCP deposit, AutoTrigger can launch workflows, and another PC can execute the work, there is no need to grant workers direct write access to live Notebooks. Shared objects can remain in SourceVault as `sv://` artifacts, snapshots, and references, while final Notebook application remains centralized in the NBAccess final action or Orchestrator committer. This is the central design principle that reconciles increasing autonomy with controlled execution boundaries.

Principal code and documentation sources:

SourceVault_info/design/sourcevault_universal_mcp_access_spec_v2.md /
SourceVault_info/docs/api_mcp.md / SourceVault_info/docs/api_autotrigger.md /
SourceVault_info/docs/api_diagnostics.md

9.27 Outlook

The highest-priority next step is to migrate LLM transmission boundaries in stages from Shadow to Warn, then to endpoint-specific Enforce, and finally to system-wide Enforce. The technical foundation is largely present, so the practical task is to widen enforcement while observing token coverage, false refusals, and unregistered entry points. If `PreparedInputTokens` are minted upstream

for every call chain and consumed immediately before provider transmission, the system becomes substantially more resistant to post-inspection substitutions of prompts, retrieval content, and tool schemas.

Next, `CapabilityLease` should be connected to every side-effect boundary, including send, publish, delete, network, secret, and Notebook write operations. Prompt injection may never be perfectly preventable, but if a deceived LLM cannot expand its own authority and can execute only an explicitly permitted operation against an explicitly permitted target through one-shot capability, the blast radius remains limited.

For taint, a major task is to record `ContributingSpanRefs` across every important path: Web -> Summary, Mail -> Answer, PDF -> Report, MCP retrieval -> Agent output, and Agent artifact -> next Agent. The more SourceVault becomes a sharing plane among agents, the more important this span-level provenance becomes.

SafeInspection and declassification workflows are also important future work. Normal inspection of quarantined payloads should prohibit copy/export, while external release should require a sanitized projection and explicit declassification. This reduces the risk of re-infection caused by the act of investigating malicious content.

Anomaly analysis should remain observe-only for the time being and can reasonably evolve toward campaign detection using correlations such as `InjectionSignalRate` with `RunIntegrityRate`, or pre-scan failures with unexpected tool requests. These correlations should not automatically cause permission restriction; the current cautious policy of requiring additional evidence such as controlled replay should be preserved.

In addition, `RoutineExecutionRate`, `RoutineOverdueRate`, and `OverdueContractSeconds` should mature as **operational anomaly streams** alongside LLM and system anomalies. The purpose is not to diagnose the owner, but to detect early that an expected `Obligation` has not been fulfilled. When `Calendar`, `Notebook`, or `Mail` source coverage is `Stale` or `Unavailable`, the system should not misclassify that as non-execution; coverage degradation must remain a separate stream.

For the unified Agenda, the R9 Mail Agenda specification should progress from draft toward a stable version, and implementation-status text in `api_routine.md` should be synchronized with the latest `main` branch. Important next issues include automatic Done detection when a reply is sent through an external mail client, formal integration of mail-thread <-> inheriting-Notebook relationships into the mining graph, conflict tolerance when shared sidecars are updated from multiple PCs, and consistent occurrence identity across Agenda, Board, and PlanView.

For PlanView, the remaining work includes wiring durable `PlanningConstraint` / `PlanningIntent` / `PlanChunkProgress` into the shared ledger, implementing the Plan diff view, and completing the Petri adapter only for formal workflow scopes. `LatestSafeStart` and `slack` are useful against procrastination, but the current principle of not silently modifying plans when inputs are stale

should remain. When a plan becomes infeasible, choices such as Defer, Waive, Automate, and NegotiateDeadline should be presented to the owner rather than silently violating constraints.

Ultimately, SystemDoctor software health, Cane LLM/security anomalies, and Routine Attention operational anomalies would benefit from a shared higher-level observability surface. They should not, however, be collapsed into a single score. Causes, privacy, authority, and recommended actions must remain distinct. This would allow the same observation infrastructure to detect very different problems - “the LLM made a mistake,” “the service is broken,” or “important work remains unexecuted” - while still routing each to a different and safe response. # Chapter 10. Comparison with Similar Systems

This chapter compares the transreal stack with systems such as LangGraph, Microsoft Agent Framework, and Haystack. A simple comparison based on “how many agent-framework features are available” would miss the stack’s defining characteristics. Its distinctiveness lies in vertically integrating Notebook cell privacy, held validation of Wolfram expressions, the single-committer model, source/snapshot/provenance management, Knowledge Home, MCP, and diagnostics.

At the same time, large Python-based open-source ecosystems have clear advantages in generality, ease of adoption, provider ecosystems, public CI, cross-platform deployment, and community support. The right comparison is therefore not “which is better?” but “what is each system optimizing for?” LangGraph is strong as a general durable agent graph; Microsoft-oriented frameworks emphasize production governance; Haystack is strong in RAG pipelines; and the transreal stack is deeply specialized for a safe knowledge-work environment centered on Wolfram Notebooks.

The addition of MCP and `sv://` URIs has improved connectivity with external models, but the center of gravity remains Notebook-based local/private knowledge. That specialization is both a constraint and the reason the stack can integrate capabilities such as cell-level privacy and source provenance that other frameworks do not normally provide by default.

10.1 Comparison Axes

When the later chapters of this report are included, multi-agent capability alone is not an adequate comparison axis.

Comparison axis	Description
Execution Control	How LLM proposals are validated and side effects constrained
Workflow	Long-running work, multi-agent execution, approval, retry
Knowledge Lifecycle	Sources, snapshots, claims, versions
Retrieval	Lexical, vector, KG, neighborhood search
Identity	Authors, senders, entity resolution

Comparison axis	Description
Privacy	Notebooks, sources, prompts, release
Security	Prompt injection, taint, capability control
Diagnostics	Probes, heartbeats, doctor
Long-term Support	Search episodes, cognition, review
Deployment	Providers, languages, platforms, CI

10.2 Comparison Table

Project	Core capabilities	Primary design focus	Adoption difficulty	Maturity	License
transreal stack	Notebook privacy, single-agent Runtime, multi-agent orchestration, source-first vault, Knowledge Home	Wolfram Notebook-centered, separation of responsibilities, single committer, hash-addressed sources, local privacy	Relatively high	Medium	MIT
LangGraph	Stateful agent orchestration	Graphs, durable state, human-in-the-loop	Medium	High	MIT
Microsoft Agent Framework	Production-grade multi-agent workflow	Python/.NET, durability, observability, governance	Medium	High	MIT
Haystack	Retrieval, pipelines, agents, RAG	Modular retrieval/routing/generation	Medium	High	Apache-2.0

10.3 Comparison with LangGraph

LangGraph is strong in long-running stateful agent workflows, checkpointing, human-in-the-loop control, branching, retries, and related capabilities.

The main difference is that LangGraph is a general Python agent runtime, while the transreal stack vertically integrates Notebooks, Wolfram expressions, cell privacy, and SourceVault.

LangGraph has advantages in providers, deployment options, and community integrations. The transreal stack, however, is distinctive in Notebook-cell privacy, held Wolfram-expression validation, the Notebook single-committer model, integrated source/snapshot/claim handling, the OOPS Knowledge Home, and the SensitiveLocalVault.

10.4 Comparison with Microsoft Agent Framework

Microsoft Agent Framework emphasizes enterprise governance, observability, Python/.NET environments, and production deployment.

The transreal stack is smaller and less mature operationally, but differs in its attempt to unify an individual's or researcher's Notebook-based work with sources, privacy, and long-term memory.

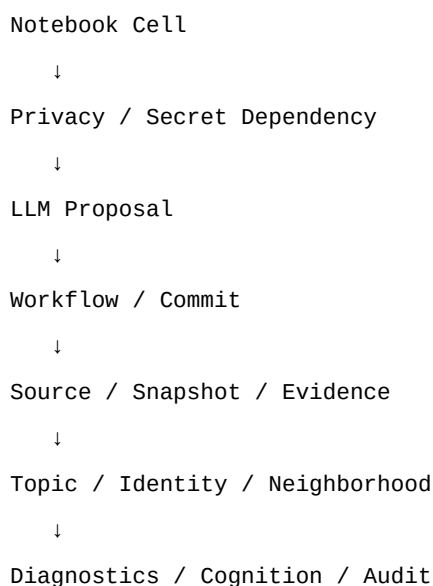
10.5 Comparison with Haystack

Haystack is a mature RAG platform that clearly composes retrievers, rankers, document stores, generators, pipelines, and related components.

SourceVault also includes search and primer mechanisms, but its emphasis is not merely document retrieval. It integrates immutable snapshots, staleness propagation, Evidence Bundles, Notebook management, identity, owner-scoped topics, quote/session graphs, and release contexts.

10.6 Positioning of the transreal Stack

Rather than being a general-purpose framework targeting exactly the same market as LangGraph, Microsoft Agent Framework, or Haystack, the transreal stack is a specialized stack vertically integrated around safe execution, knowledge management, and long-term assistance in Wolfram Notebooks.



This end-to-end integration is its greatest point of distinction.

10.7 Outlook

Future comparisons should extend beyond “agent frameworks” to include personal knowledge management, secure RAG, workflow automation, and MCP data servers. SourceVault’s functional scope is now broad enough that no single category of competitor fully explains its positioning.

Interoperability with external frameworks is also a promising practical direction. For example, LangGraph or another agent framework could use SourceVault through MCP, leaving knowledge, privacy, and provenance to SourceVault while external frameworks provide agent orchestration. This would let the transreal stack expose its strongest components as services rather than forcing them into a closed monolith.

More quantitative evaluation is also needed to make comparisons persuasive. Public benchmarks corresponding to the system’s design claims - workflow success rates, Notebook corruption rates, privacy-leak tests, retrieval groundedness, multi-PC recovery time, prompt-injection containment rate, and similar measures - would make comparisons with mature open-source systems more objective. # Chapter 11. Maturity, Open Issues, and Future Priorities

This system contains a mixture of fully implemented areas, implemented areas that are still in staged migration, observe-only areas, and areas where design leads implementation. It is therefore not enough to say simply that “SourceVault has this feature”; any description should also state which mode is the default and how far end-to-end integration has progressed.

For example, OOPS/Knowledge Home phases 1A-1C are implemented and verified, but unified cross-media NeighborhoodSearch is still under integration. Capability Broker is implemented, but the default LLM-boundary mode is Shadow. AutoTrigger supports `specificMachine` remote runs, while WorkerPool automatic load balancing and failover remain incomplete. Understanding these differences in maturity across subsystems is essential for realistic operational planning.

The maturity of a public open-source project is also a separate axis from the sophistication of its internal capabilities. Without CI, releases, version matrices, cross-platform testing, threat models, and migration guides, reproducibility remains low for external users. The system should therefore be evaluated separately as an advanced research/development platform and as generally distributable software.

11.1 Avoid Treating Maturity as a Single Score

Area	Assessment
Core NBAccess privacy and held-expression validation	Core implementation exists
Runtime proposal loop	Core implementation exists
Orchestrator planner/spawn/reduce/commit	Core implementation exists
Petri-net workflows	Implemented
Source/Snapshot/Bundle	Implemented

Area	Assessment
PromptRouter/Release Context	Implemented
OOPS seed/Knowledge Home 1A-1C	Implemented and verified
General mail structuring	Implemented, still expanding
Unified cross-media Neighborhood	Partially wired
Capability Broker	Implemented, staged migration underway
Cognitive Support	Shadow-first
Anomaly analysis	Observe-only
Public CI/releases	Limited
macOS/Linux	Insufficient public verification

11.2 Scale of SourceVault Modules

SourceVault has become highly modular and now consists of an umbrella loader plus many subpackages.

Long-term maintenance will increasingly depend on automated verification of the inter-package dependency DAG, separation of public and private APIs, tests of initialization order, graceful degraded operation when optional modules are absent, contract tests, schema migration, event replay tests, snapshot compatibility, and prevention of accidental mixing among source, derived, and projection objects.

11.3 Security Priorities

For NBAccess, key priorities include confidential dependency propagation, the `CloudPublishable` boundary, held-expression bypass resistance, cross-kernel consistency of the key index, and MAC/tamper tests.

For Runtime/Orchestrator, priorities include adapter bypass, worker deny-heads, committer capabilities, duplicate execution on retries, external-process cleanup, and policy re-validation after approval.

For SourceVault, priorities include request-time release gates, revocation, leakage of private topic labels, substitution of prompt/retrieval envelopes, directionality of taint edges, leakage through heartbeat fields, and deterministic event replay.

11.4 Challenges as Public Open Source

From the perspective of the public repositories, further work is needed on public CI such as GitHub Actions, test-coverage metrics, tag/release operations, a version-compatibility matrix, validation beyond Windows, reproducible setup, a minimal sample vault, standalone documentation of the security model, a threat model, migration guides, and a public changelog.

11.5 Recommended Staged Consolidation

Stage 1: Stabilize the Four Layers

- NBAccess boundary tests
- Runtime adapter contracts
- Orchestrator commit tests
- SourceVault core replay tests

Stage 2: Integrate Knowledge Home

- Common TopicPosition across Mail, PDF, Notebook, and Web
- SharedAuthor / SharedTag edges
- Unified NeighborhoodSearch
- Cross-media live view

Stage 3: Move Safety Boundaries into Enforcement

- Move LLM boundary from Shadow to Warn
- Enforce selected entry points
- Integrate CapabilityLease with dispatch
- Enforce sink field allowlists

Stage 4: Evaluate Long-Term Knowledge

- Measure the effectiveness of retrieval episodes
- Evaluate OOPS seed recall/NDCG
- Measure novel-topic growth rates
- Measure false-link/false-topic rates
- Measure false-intervention rates for cognitive support

11.6 Outlook

In the short term, prioritizing boundary tests for the four layers and public CI is the most rational strategy. Fixing the core boundaries - NBAccess privacy, Runtime adapters, Orchestrator commit, and SourceVault replay - before further expansion will reduce long-term regression risk.

In the medium term, unified cross-media Knowledge Home, LLM-boundary Enforce, CapabilityLease integration with actions, automatic triggering of a comprehensive SystemDoctor, and multi-PC WorkerPool scheduling would bring the currently distributed advanced features into a more coherent operational experience.

In the long term, the project should establish public benchmarks, a sample vault, reproducible setup, and a threat model, thereby advancing from research code toward a reproducible platform. Claims about safety, in particular, should be supported by continuous testing that includes attack tests and failure injection. # Chapter 12. Overall Assessment

The four repositories have evolved from “tools for calling LLMs from Mathematica” into “an environment centered on the Notebook in which knowledge is acquired, evidence is preserved, agents reason, workflows execute, and resulting artifacts are accumulated again as knowledge.”

What creates the system’s distinctiveness is not the number of features but the design of its boundaries. Secrets are assigned to NBAccess, the single turn to Runtime, multi-agent orchestration and commit to Orchestrator, and source lifecycles and knowledge space to SourceVault. Even as MCP and AutoTrigger extend the system toward external clients and autonomous execution, the architecture attempts to preserve authority and provenance boundaries.

At the same time, this is not a finished monolithic product. Unified cross-media neighborhood search, system-wide Enforce, WorkerPool failover, public CI, and other areas remain future work. The most reasonable assessment is therefore that it is an advanced research-and-development platform with a forward-looking architecture and substantial implementation across its major components.

12.1 Basic Assessment

The four repositories have had clearly separated responsibilities from the beginning.

- NBAccess: the boundary for Notebooks and secrets
- ClaudeRuntime: the execution kernel for a single turn
- ClaudeOrchestrator: multiple workers and commit discipline
- SourceVault: source lifecycle and deterministic replay

In particular, several design choices are consistently aimed at reducing operational accidents: workers are prohibited from writing directly to Notebooks and commit is centralized in a single committer; PromptRouter returns only unevaluated expressions and does not evaluate them itself; and Runtime delegates safety decisions to NBAccess.

12.2 Assessment Including the Later Layers

With the addition of the SourceVault mining layer, the system can do more than store and search materials. It can reorganize knowledge through topics, identity, quotations, sessions, and work history.

The OOPS corpus is not merely historical mail.

Approximately 6,500 messages are accompanied by about 4,100 manually constructed topic items, topic relations, mail-to-topic mappings, and quotation relations. This structure provides an initial coordinate system for positioning new general mail, Notebooks, PDFs, and web pages.

At the same time, the layered safety architecture controls risks introduced by broader connectivity among knowledge objects.

- Privacy is protected through NBAccess and release contexts.
- Prompt injection is isolated through untrusted-data treatment and taint.
- LLM transmissions are bound through PreparedInputToken.

- External actions are constrained by `CapabilityLease`.
- Notebook commit is limited to the single committer.
- Diagnostic information is constrained through sink allowlists.
- Cognitive support is local-only and shadow-first.
- Anomaly analysis begins in observe-only mode.

This architecture separates “being near in search,” “being readable,” “being releasable to an LLM,” and “being executable as an external action.”

12.3 Technical Value

Safe connection between confidential Notebooks and LLMs. The system identifies secret Notebook cells and their dependent outputs, varies context by destination, and validates LLM proposals as held expressions.

Source-centric reproducibility. It tracks sources, snapshots, spans, claims, workflows, and generated artifacts and can assess artifact staleness when upstream materials change.

A personal Knowledge Home. Starting from the material currently being viewed, the user can move toward the next relevant piece of knowledge through topics, people, quotations, time, and past exploration.

12.4 Constraints

For users seeking a general-purpose replacement for Python agent frameworks, systems such as LangGraph or Haystack may be easier to adopt.

The transreal stack assumes Mathematica, Wolfram Language, Notebooks, multiple package loads, and a local vault.

Its maturity in public CI, releases, and platform validation also remains limited.

Further integration and evaluation are still needed for fully cross-media Knowledge Home, system-wide enforcement of safety boundaries, and the real-world effectiveness of cognitive and anomaly functions.

12.5 Final Conclusion

Rather than describing the system as a finished general-purpose agent product, the following assessment is more appropriate.

NBAccess, ClaudeRuntime, ClaudeOrchestrator, and SourceVault form an advanced and substantially implemented research-and-development platform that integrates confidentiality boundaries, LLM reasoning, multiple agents, source lifecycles, knowledge graphs, neighborhood exploration, execution authority, and auditing around the Mathematica Notebook.

The four repositories have moved beyond the scope of “tools for calling LLMs from Notebooks” and are evolving into a system that manages knowledge acquisition, structuring, exploration, reasoning, execution, and auditing under one model of separated responsibilities.

Their distinctiveness lies not in the number of individual features, but in treating the following entire sequence as one continuous model.

```

Notebook
  ↓
Privacy
  ↓
LLM Proposal
  ↓
Validation
  ↓
Workflow
  ↓
Commit
  ↓
Source and Evidence
  ↓
Topic and Identity
  ↓
Neighborhood
  ↓
Diagnostics and Audit
```

This is an attempt to reframe the Wolfram Notebook from a computational document into a working environment for long-term knowledge formation, reasoning, execution, and verification.

12.6 Outlook

The area with the greatest potential for future growth is an architecture in which multiple LLMs, multiple PCs, and multiple workflows sharing the same SourceVault also share the same `sv://` object graph and policy model. Humans would continue to use the Notebook as the primary UI, while local LLMs, cloud LLMs, remote workers, and scheduled workflows cooperate in the background.

To realize that future safely, `PreparedInputToken`, `CapabilityLease`, the single-committer model, release gates, and taint provenance need to become standard execution paths rather than optional features. As autonomy increases, authority separation and auditability must be strengthened in the opposite direction.

Ultimately, this leads toward treating the Wolfram Notebook not merely as a computational document but as a personal or research-group **Knowledge Operating Environment**. Sources, conversations, code, specifications, workflows, mail, documents, and experimental results would be connected in the same reference space and reused while preserving their individual privacy and provenance. This appears to be the long-term destination of the four-repository stack.

Appendix A. Principal References

This report was organized primarily from the README files, API documentation, design specifications, and major implementation files of the public `transreal` GitHub repositories as of July 15, 2026. The later chapters relied especially heavily on the following materials.

- `NBAccess/README.md`
- `NBAccess.wl`
- `NBAccess_crypto.wl`
- `NBAccess_info/design/ClaudeEval_runtime_orchestrator_nbaccess_async_compat_final_spec_permission_modes_v3.md`
- `ClaudeRuntime/README.md`
- `ClaudeRuntime.wl`
- `ClaudeRuntime_processsupervisor.wl`
- `ClaudeRuntime_info/docs/api_processsupervisor.md`
- `ClaudeOrchestrator/README.md`
- `ClaudeOrchestrator.wl`
- `ClaudeOrchestrator_workflow.wl`
- `ClaudeOrchestrator_info/docs/api.md`
- `SourceVault/README.md`
- `SourceVault_core.wl`
- `SourceVault_searchindex.wl`
- `SourceVault_identity.wl`
- `SourceVault_mining.wl`
- `SourceVault_oopsseed.wl`
- `SourceVault_mailstructure.wl`
- `SourceVault_knowledgehome.wl`
- `SourceVault_diagnostics.wl`
- `SourceVault_capbroker.wl`
- `SourceVault_taint.wl`
- `SourceVault_cognition.wl`
- `SourceVault_adjudication.wl`
- `SourceVault_anomaly.wl`
- `SourceVault_info/design/sourcevault_search_foundation_implementation_spec_v1.md`
- `SourceVault_info/design/sourcevault_self_organizing_mining_identity_tag_spec_v0_1.md`
- `SourceVault_info/design/sourcevault_cane_knowledge_home_mining_spec_v0_7.md`
- `SourceVault_info/design/sourcevault_cane_handoff_2026-07-14.md`
- `SourceVault_info/docs/api_knowledgehome.md`

- SourceVault_info/docs/api_oopsseed.md
- SourceVault_info/docs/api_identity.md
- SourceVault_info/docs/api_cognition.md

Descriptions of implementation status distinguish between the intent stated in design specifications and the implementation state verifiable from public code, API documentation, and handoff materials.

Appendix B. Principal Code, API, and Design Documents Consulted for This Expansion

This expanded edition cross-checked not only README files, but also implementation code publicly available on GitHub as of July 16, 2026, API references under SourceVault_info/docs, design specifications under SourceVault_info/design, and implementation specifications for coded workflows. The following are the principal sources supporting the added material.

Area	Principal reference files
claudecode foundation	transreal/claudecode/claudecode_info/docs/api.md; claudecode_info/docs/api_directives.md
PromptRouter	SourceVault_info/docs/api_promptrouter.md; SourceVault_promptrouter.wl
PromptWorkflow	ClaudeOrchestrator_promptworkflow.wl
Requirements -> specification	SourceVault_workflows/spec-review/ SVWorkflow_SpecReview_info/design/ implementation-spec.md
Specification -> implementation	SourceVault_workflows/spec-impl/ SVWorkflow_SpecImpl_info/design/ implementation-spec.md
Workflow registry	SourceVault_info/docs/api_workflowregistry.md
Practical SourceVault use	SourceVault_info/docs/user_manual.md
sv:// URI / MCP	SourceVault_info/docs/api_mcp.md; SourceVault_mcp.wl; SourceVault_info/design/sourcevault_universal_ mcp_access_spec_v2.md
AutoTrigger	SourceVault_autotrigger.wl; SourceVault_info/docs/api_autotrigger.md; SourceVault_info/design/sourcevault_auto_trigger_scheduler_spec_v0_1.md
Diagnostics / SystemDoctor	SourceVault_info/docs/api_diagnostics.md;

Area	Principal reference files
	<code>SourceVault_diagnostics.wl</code>
RuntimeSession / artifact boundary	<code>ClaudeOrchestrator_info/design/</code> <code>claude_orchestrator_runtime_session_episode_pe</code> <code>tri_spec_v0_1.md</code>
	• <code>SourceVault_info/design/sourcevault_routine_attention_spec_v0_4.md</code> - Canonical Routine Attention core specification. v0.3 is retained for historical reference. • <code>SourceVault_info/design/sourcevault_routine_planview_ext_spec_v0_1.md</code> - Planning and visualization extension. • <code>SourceVault_info/design/sourcevault_routine_mail_agenda_spec_v0_1.md</code> - R9 Mail Agenda specification. • <code>SourceVault_info/docs/api_routine.md</code> / <code>SourceVault_routine.wl</code> - Obligation, Attention, R7 anomaly streams, R8 automation. • <code>SourceVault_info/docs/api_routineplan.md</code> / <code>SourceVault_routineplan.wl</code> - Plan, ScheduleFabric, Agenda, visualization. • <code>SourceVault_mailagenda.wl</code> - Actionable mail, resolution state, mail <-> Notebook inheritance. • Recent implementation commits checked: 409012e (R7/R8/AT-1/routineplan core), c0c487d (Agenda/visualization/ScheduleFabric/replan), c8d6e82 (R9 mailagenda integration).

Interpreting Implementation Status

This report distinguishes between a feature being present in a specification and being implemented in the current code. In particular, three points are explicitly separated to avoid overstatement: (1) the policy that a comprehensive SystemDoctor should be the “first automatic workflow” is canonical in the AutoTrigger design specification, but its automatic triggering remains deferred in the diagnostics API; (2) remote execution through `specificMachine` is implemented, while `WorkerPool` automatic load balancing/failover is not; and (3) `sv://` is the canonical external identity layer, while legacy internal references remain for compatibility.

The possible future split into `claudecode_core` is likewise described not as an already implemented fact in public code, but as a future architectural expectation for separating the current responsibilities of `claudecode.wl` into a core and a legacy compatibility layer.